
Process Scheduling

Single and Dual Processor Simulators

A comparative study of FIFO, Round Robin and
Multi-Level Feedback Queue scheduling

Submitted by

Ayush Yadav – CS24BT001

Trisham Bepari – CS24BT022

Abstract. Discrete-event CPU scheduling simulators for one and for two processors were implemented in C++ and evaluated on five workloads. Four policies were compared: FIFO, Round Robin, MLFQ without priority boost and MLFQ with a priority boost every 20 time units. Turnaround time, CPU utilisation and fairness were measured for every process in every run, and the time quantum and boost interval were swept. On one processor FIFO achieved the lowest average turnaround time on every workload (tied on one), while the preemptive policies were far better on worst-case turnaround, fairness and CPU utilisation. Adding a second processor reduced average turnaround time by 7–50 % and shortened the makespan by 1.14–2.00×, narrowed the differences between the policies, and changed the winner on one workload, where Round Robin overtook FIFO.

Contents

1	Introduction and Objective	4
1.1	Workload file format	4
1.2	Metrics reported	4
2	Simulator Design	5
2.1	Single processor	5
2.2	Dual processor	5
2.3	Correctness checks	5
3	Theory of the Scheduling Algorithms	6
3.1	FIFO (First In First Out / FCFS)	6
3.2	Round Robin (RR)	6
3.3	MLFQ without priority boost	7
3.4	MLFQ with periodic priority boost	7
3.5	Scheduling on two processors	8
4	How to Build and Run	9
4.1	Running with the makefile	9
4.2	Running without the makefile	10
4.3	Understanding the output	11
5	The Workloads	13
I	Single Processor	14
6	Detailed Turnaround Time Calculations	14
6.1	Workload 1	14
6.2	Workload 2	14
6.3	Workload 3	15
6.4	Workload 4	15
6.5	Workload 5	15
7	Consolidated Summary	16
8	Comparative Graphs	16
8.1	Per-process turnaround time	16
8.2	Schedule visualisation (Gantt charts)	20
9	Additional Experiments	23
9.1	Effect of the Round Robin time quantum	23
9.2	Effect of the MLFQ priority boost interval	23
10	Findings and Analysis	25
10.1	Finding 1: FIFO has the lowest <i>average</i> turnaround time	25
10.2	Finding 2: FIFO loses badly on <i>worst-case</i> turnaround time	26
10.3	Finding 3: preemption dramatically improves CPU utilisation	26
10.4	Finding 4: the priority boost trades worst case for average	26
10.5	Finding 5: neither parameter is monotonic	27
10.6	Finding 6: when all four algorithms coincide	27
10.7	Finding 7: the hidden cost of preemption	27
11	Summary of Part I	28

II	Dual Processor	29
12	Overview of Part II	29
13	Detailed Turnaround Time Calculations	29
13.1	Workload 1	29
13.2	Workload 2	30
13.3	Workload 3	30
13.4	Workload 4	30
13.5	Workload 5	31
14	Consolidated Summary	32
15	Comparative Graphs	32
15.1	Per-process turnaround time	34
15.2	Schedule visualisation (Gantt charts)	37
16	Additional Experiments	41
16.1	Effect of the Round Robin time quantum	41
16.2	Effect of the MLFQ priority boost interval	42
17	Findings and Analysis	43
17.1	Finding 1: FIFO is no longer best everywhere	43
17.2	Finding 2: the convoy effect is much weaker	44
17.3	Finding 3: where FIFO wastes processor time	44
17.4	Finding 4: the priority boost no longer helps	44
17.5	Finding 5: the parameters matter much less, but are still non-monotonic	45
17.6	Finding 6: when all four algorithms coincide	45
17.7	Finding 7: preemption now also causes migration	45
17.8	Finding 8: the two processors are not equally loaded	46
18	Summary of Part II	46
III	Single versus Dual Processor	47
19	Head-to-Head Results	47
19.1	Turnaround time	47
19.2	Makespan, utilisation and overhead	48
19.3	Ranking of the algorithms	48
19.4	How much the choice of algorithm matters	48
20	Comparative Graphs	49
20.1	Per-process improvement	49
20.2	The same workload on one and two processors	54
21	Findings of the Comparison	55
21.1	A second processor helps every policy on every workload	55
21.2	The speedup is almost always below 2	55
21.3	The preemptive policies gain more than FIFO	55
21.4	The ranking changes and the gaps narrow	55
21.5	Utilisation falls: diminishing returns	56
21.6	The convoy effect largely disappears	56
21.7	The priority boost stops paying off	56
21.8	Parameter tuning matters less	56

21.9 Fewer dispatches, but a new overhead	56
22 Overall Summary and Conclusion	57

1 Introduction and Objective

This report accompanies two discrete-event CPU scheduling simulators written in C++: one for a *single* processor and one for *two* processors. Each simulator reads a workload description file, simulates a chosen scheduling algorithm, and reports both the resulting schedule and a set of performance metrics.

The algorithms studied are:

1. **FIFO** (First In First Out, also called FCFS).
2. **Round Robin** with a configurable time quantum.
3. **MLFQ** (Multi-Level Feedback Queue) with three queues and no priority boost.
4. **MLFQ with periodic priority boost** every 20 time units.

The objectives are, first, to measure how each policy affects *turnaround time*, *CPU utilisation* and *fairness* on one processor and explain the differences in terms of how each policy makes its decisions; and second, to re-evaluate all the policies on two processors and determine what a second processor changes.

The report is organised as follows. Sections 1–5 cover material common to both simulators: metrics, design, theory, build instructions and workloads. **Part I** presents the single-processor results, **Part II** the dual-processor results, and **Part III** compares the two directly.

1.1 Workload file format

Each line of a workload file describes one process:

```
<arrival-time> <cpu-burst-1> <io-burst-1> <cpu-burst-2> <io-burst-2> ... -1
```

For example the line 0 100 2 200 3 25 -1 means: arrival time = 0; CPU burst 1 = 100; I/O burst 1 = 2; CPU burst 2 = 200; I/O burst 2 = 3; CPU burst 3 = 25; end of process. Every line is terminated by -1, a process may have any number of CPU/I/O burst cycles, and arrival times are non-decreasing.

1.2 Metrics reported

Turnaround time (TAT) For a process P_i with arrival time A_i and finish time F_i , the turnaround time is

$$\text{TAT}_i = F_i - A_i.$$

This includes CPU time, I/O time and all time spent waiting in a ready queue.

Average turnaround time $\frac{1}{n} \sum_{i=1}^n \text{TAT}_i$.

Maximum turnaround time $\max_i \text{TAT}_i$ — the worst-case latency, a measure of starvation.

Minimum turnaround time $\min_i \text{TAT}_i$ — the best-served process.

Run time (not counting I/O) The number of time units during which a CPU was actually executing a process. On two processors this is reported for each CPU and as a total. Because I/O is overlapped with computation, run time is *not* the same as the total simulation time.

Total simulation time (makespan) The time at which the last process finishes.

CPU utilisation $\frac{\text{run time}}{N \times \text{makespan}}$, where N is the number of processors. The remainder is idle CPU time.

Speedup $\frac{\text{makespan on one processor}}{\text{makespan on two processors}}$. A speedup of $2\times$ would mean the second processor halved the total time.

2 Simulator Design

The simulators advance in discrete time units (“ticks”). Each tick is processed in a fixed four-phase order, which is identical for all four algorithms and for both simulators:

1. **Arrivals.** Any process whose arrival time equals the current time is placed in the ready structure.
2. **Dispatch.** Every idle processor selects a process according to the scheduling policy.
3. **Execute.** Every running process consumes one unit of its CPU burst and every process in the I/O queue consumes one unit of its I/O burst.
4. **Record, then apply transitions.** The state of the tick is recorded, the clock is advanced, and only *then* are state changes applied (burst completion, I/O completion, quantum expiry, priority boost).

Performing the transitions *after* recording the tick is essential for correctness of the reported schedule: a process that finishes its CPU burst during tick t genuinely occupied a CPU for that tick and must be shown as running, not as already blocked on I/O or already terminated.

2.1 Single processor

The single-processor simulator keeps one `running_pid`, one time-slice counter and, for MLFQ, the level of the running process. The ready structure is a single FIFO queue for FIFO and Round Robin, and three queues Q_0, Q_1, Q_2 for MLFQ.

2.2 Dual processor

The dual-processor simulator is the same program with the per-processor state turned into arrays of size two: `running_pid[2]`, `process_time_slice[2]` and `running_level[2]`. The dispatch, execute and transition phases each loop over the two processors. The key design decisions are:

- **One shared ready queue.** Both processors take work from the same global queue (or the same Q_0, Q_1, Q_2 for MLFQ). A processor can therefore never be idle while a runnable process is waiting.
- **Deterministic tie-breaking.** CPU0 always dispatches before CPU1, and when both processors change state in the same tick, CPU0’s transition is applied first. The simulation is therefore fully repeatable.
- **Migration is allowed.** A preempted process returns to the shared queue and may resume on either processor.
- **Identical policy parameters.** The quantum (2), the boost interval (20) and all other rules are exactly those of Part I, so every difference between the two parts is caused by the extra processor alone.

2.3 Correctness checks

Both simulators were validated against an independent checker which verifies, for every run, that:

- the total CPU time recorded for each process equals the sum of its CPU bursts;
- the total I/O time recorded for each process equals the sum of its I/O bursts;
- no processor runs more than one process in a tick, and no process runs on two processors in the same tick;

- no process executes before its arrival time;
- a terminated process never becomes runnable again;
- no processor is idle while the ready queue is non-empty;
- the gap between consecutive CPU bursts of a process is at least the intervening I/O burst length;
- the printed metrics agree with values recomputed from the printed schedule.

Every algorithm/workload combination reported here passes every check. Two further checks were applied to the dual-processor simulator:

- **Reduction to one processor.** Compiling the dual-processor program with the processor count set to 1 reproduces the output of the single-processor simulator byte for byte, in all 110 configurations tested. The two programs therefore differ only in the number of processors.
- **Hand calculation.** Workload 1 under FIFO on two processors was worked out by hand: P1 runs on CPU0 for 200 units, P2 on CPU1 for 100 units, and P3 (arriving at 10) then runs on CPU1 from 100 to 400. This gives turnaround times of 200, 100 and 390, an average of 230 and a makespan of 400 — exactly what the simulator reports.

3 Theory of the Scheduling Algorithms

3.1 FIFO (First In First Out / FCFS)

FIFO keeps a single ready queue. A process is appended to the tail of the queue when it arrives, or when it returns from I/O. The scheduler always dispatches the process at the head of the queue and lets it run *to the end of its current CPU burst*. FIFO is therefore **non-preemptive**: once a process is given the CPU it keeps it until it blocks for I/O or terminates.

Properties.

- **Simple and cheap.** One queue, no timers, minimum number of context switches.
- **No starvation.** Every process eventually reaches the head of the queue.
- **Convoy effect.** A long CPU-bound process at the head of the queue forces every short process behind it to wait. Short, interactive jobs suffer badly.
- **Poor overlap of CPU and I/O.** While a long burst runs, I/O-bound processes sit in the ready queue instead of issuing their I/O requests early. Conversely, once the CPU-bound process finally blocks, the CPU may go idle because all the other processes are simultaneously blocked on I/O.
- **Bad response time.** There is no notion of priority or of recent behaviour.

3.2 Round Robin (RR)

Round Robin uses the same single FIFO ready queue, but adds a **time quantum** q . A dispatched process runs until the earliest of: (a) its CPU burst completes, (b) it has run for q consecutive time units. In case (b) the process is preempted and appended to the *tail* of the ready queue, and the next process is dispatched.

Properties.

- **Preemptive and fair in the short term.** With n runnable processes no process waits more than roughly $(n - 1)q$ time units for the CPU, so response time is bounded.
- **Solves the convoy effect.** A long burst can no longer monopolise the CPU.
- **Choice of q is a trade-off.** If q is very small, response time is excellent but context-switch overhead dominates. If q is very large, RR degenerates towards FIFO.
- **Average turnaround time is often worse than FIFO.** Because RR interleaves all jobs, processes tend to finish close together near the end of the schedule, instead of some finishing very early.
- **Treats all processes identically.** It has no way to favour short or interactive jobs, because it does not observe past behaviour.

3.3 MLFQ without priority boost

MLFQ addresses the weakness that RR treats all jobs alike. It maintains three queues, Q_0 (highest priority), Q_1 and Q_2 (lowest), each scheduled Round Robin with a time quantum of 2. The rules are:

1. A new process enters at the highest priority queue Q_0 .
2. The scheduler always dispatches from the highest non-empty queue, i.e. Q_0 before Q_1 before Q_2 .
3. If a process uses up its entire time quantum, it is **demoted** one level ($Q_0 \rightarrow Q_1 \rightarrow Q_2$; a process already in Q_2 stays in Q_2).
4. If a process blocks for I/O before its quantum expires, it **keeps** its current priority level and re-enters the same queue when its I/O completes.

Rationale. The policy *learns* the behaviour of a process without being told anything in advance. A process that repeatedly gives up the CPU quickly is almost certainly interactive or I/O-bound, and it stays near the top. A process that always exhausts its quantum is CPU-bound and sinks to the bottom, where it still runs, but only when nothing more urgent is ready. This approximates “shortest job first” behaviour using only observed history.

Weakness: starvation. If enough high-priority work keeps arriving, a process that has sunk to Q_2 may never be scheduled again. Furthermore a process can *game* the scheduler: by issuing an I/O operation just before its quantum expires it retains high priority indefinitely while still consuming almost a full quantum of CPU.

3.4 MLFQ with periodic priority boost

To repair the starvation weakness, a **priority boost** is applied periodically: every B time units (here $B = 20$), *every* process in the system is moved back to the highest priority queue Q_0 , and the currently running process is also returned to Q_0 .

Properties.

- **Guarantees no starvation.** Any process waits at most B time units before being restored to the top queue.
- **Repairs misclassification.** A process whose behaviour changes from CPU-bound to interactive is re-evaluated after each boost.

- **Cost.** Boosting discards the information the scheduler has learned. Long CPU-bound jobs are repeatedly promoted back above short jobs, which can *increase* the worst-case turnaround time of the short jobs.
- **Choice of B is a trade-off.** Too small, and MLFQ degenerates into plain Round Robin because demotion information is erased before it can be used. Too large, and starvation protection becomes ineffective.

3.5 Scheduling on two processors

With more than one processor the scheduler must decide not only *which* process runs next but also *where*.

Shared versus per-processor queues. The simplest design, used here, is a single *global* ready queue from which every idle processor takes work. It is *work-conserving*: a processor is never idle while a process is waiting. The alternative, one queue per processor, scales better on large machines because processors do not contend for one queue, but it needs a load-balancing policy to move work from busy to idle processors.

How each policy generalises.

- **FIFO:** the two processors serve the head of the same queue; each process still runs its burst to completion on the processor that picked it up.
- **Round Robin:** each processor keeps its own quantum counter, and a preempted process rejoins the tail of the shared queue.
- **MLFQ:** each idle processor dispatches from the highest non-empty shared queue; demotion is decided by the processor that ran the process; a boost returns the processes running on *both* processors to Q_0 .

Migration and affinity. With a shared queue a preempted process may resume on a different processor. In a real system this discards warm cache and TLB state, so operating systems prefer to keep a process on the same processor (*processor affinity*). The simulator treats migration as free, as it treats context switches.

Limits on speedup. Two processors can at most halve the makespan, and usually do not, for three reasons:

- **Too few runnable processes.** A single process can only use one processor at a time; if fewer than two processes are runnable, a processor sits idle.
- **I/O.** Time spent blocked on I/O is not reduced by extra processors.
- **Arrival times.** A process cannot start before it arrives, however many processors are free.

4 How to Build and Run

The submission has the following layout:

```
src/
  single-processor.cpp      single-processor simulator (Part I)
  dual-processor.cpp        dual-processor simulator (Part II)
  makefile                  build/run rules
CS24BT001-CS24BT022_Assignment3_report.pdf  this report
```

The five workload description files provided with the assignment are not part of the submission. The makefile expects them, named `process1.txt` to `process5.txt`, in a `processes/` directory placed next to `src/`:

```
processes/
  process1.txt ... process5.txt      workload description files
src/
  ...
```

Important: the run commands are issued from inside the `src/` directory, because the workload files are referred to with the relative path `../processes/`.

4.1 Running with the makefile

Change into the source directory first:

```
cd src
```

The makefile defines eight run targets — one per algorithm for each simulator — plus a `clean` target. Each run target automatically compiles the required binary first if it is missing or older than its source file, so no separate build step is needed.

Single processor (Part I).

```
make run-fifo-single          # FIFO
make run-round-robin-single   # Round Robin
make run-mlfq-no-boost-single # MLFQ, boost disabled
make run-mlfq-with-boost-single # MLFQ, boost every 20 time units
```

Dual processor (Part II).

```
make run-fifo-dual           # FIFO
make run-round-robin-dual    # Round Robin
make run-mlfq-no-boost-dual  # MLFQ, boost disabled
make run-mlfq-with-boost-dual # MLFQ, boost every 20 time units
```

Clean up.

```
make clean                  # delete both compiled binaries
```

Four variables control the runs. They apply to both simulators, and any of them may be overridden on the `make` command line:

Variable	Default	Meaning
FILE_NUMBER	1	Selects the workload <code>../processes/process<N>.txt</code> .
TIME_QUANTUM	2	Round Robin time quantum.
BOOST_INTERVAL	20	Priority boost period for the <code>with-boost</code> targets.
TRACE	(empty)	Set to <code>--trace</code> to additionally print the tick-by-tick state table.

Examples.

```
# FIFO on workload 4, one processor and then two processors
make run-fifo-single FILE_NUMBER=4
make run-fifo-dual FILE_NUMBER=4

# Round Robin on workload 5 with a time quantum of 8, on two processors
make run-round-robin-dual FILE_NUMBER=5 TIME_QUANTUM=8

# MLFQ without boost on workload 2, one processor
make run-mlfq-no-boost-single FILE_NUMBER=2

# MLFQ on workload 4 boosting every 10 time units instead of 20, two processors
make run-mlfq-with-boost-dual FILE_NUMBER=4 BOOST_INTERVAL=10

# FIFO on workload 3, two processors, also printing the tick-by-tick trace
make run-fifo-dual FILE_NUMBER=3 TRACE=--trace

# Remove the binaries
make clean
```

4.2 Running without the makefile

Step 1 — compile. From inside `src/`:

```
g++ -g single_processor.cpp -o single_processor.out
g++ -g dual_processor.cpp -o dual_processor.out
```

Step 2 — run. Both programs accept exactly the same command line:

```
./single_processor.out <scheduling-algorithm> <path-to-workload-file> [option] [--trace]
./dual_processor.out   <scheduling-algorithm> <path-to-workload-file> [option] [--trace]
```

Argument	Meaning
<scheduling-algorithm>	One of FIFO, RR or MLFQ (case-insensitive).
<path-to-workload-file>	Path to any workload description file. It may be relative or absolute; it is not restricted to the <code>processes/</code> directory.
[option]	An optional integer whose meaning depends on the algorithm: for RR it is the <i>time quantum</i> (default 2); for MLFQ it is the <i>priority boost interval</i> , where 0 disables boosting entirely (default 0). It is ignored by FIFO.
[--trace]	Optional. Prints the tick-by-tick state table before the schedule.

Step 3 — the four algorithms explicitly.

```
# Single processor, workload 4
./single_processor.out FIFO ../processes/process4.txt
./single_processor.out RR   ../processes/process4.txt 2
./single_processor.out MLFQ ../processes/process4.txt 0    # boost disabled
./single_processor.out MLFQ ../processes/process4.txt 20    # boost every 20

# Dual processor, workload 4
./dual_processor.out FIFO ../processes/process4.txt
./dual_processor.out RR   ../processes/process4.txt 2
./dual_processor.out MLFQ ../processes/process4.txt 0    # boost disabled
./dual_processor.out MLFQ ../processes/process4.txt 20    # boost every 20
```

Note that the two MLFQ experiments use the *same* algorithm name; the presence or absence of boosting is decided purely by the numeric option, which is exactly what the makefile targets do internally.

Further examples.

```
# A workload file stored anywhere on disk
./dual_processor.out FIFO /home/user/mytests/custom_workload.txt

# Round Robin with a large quantum (behaves almost like FIFO)
./single_processor.out RR ../processes/process5.txt 16

# MLFQ boosting very aggressively, two processors
./dual_processor.out MLFQ ../processes/process5.txt 5

# Save a run to a file
./dual_processor.out MLFQ ../processes/process4.txt 20 > result.txt
```

4.3 Understanding the output

Single processor. The program prints the schedule first, followed by the metrics:

```
CPU0
P1,1    1      30
P2,1    31      40
P3,1    41      43
P1,2    44      73
...

Average Turnaround Time: 123.00
Maximum Turnaround Time: 163
Run Time (not counting I/O): 150
Total Simulation Time: 178
```

The schedule block begins with the processor identifier `CPU0`. Each subsequent line is one contiguous stretch of CPU time and has the form

```
P<process-id>,<cpu-burst-number>      <start-time>      <end-time>
```

separated by tab characters, with start and end times *inclusive* and numbered from 1. Thus `P1,1 1 30` means “the first CPU burst of process 1 occupied the CPU from time unit 1 to time unit 30 inclusive”. If a burst is preempted and later resumed, the same burst number appears more than once — this is how preemption is visible in the output. A gap between the end of one line and the start of the next indicates an **idle CPU**.

Dual processor. The output has one schedule block per processor, each preceded by a blank line, followed by the metrics, which include the run time of each processor:

```
CPU0
P1,1    1      200

CPU1
P2,1    1      100
P3,1    101    400

Average Turnaround Time: 230.00
Maximum Turnaround Time: 390
Run Time (not counting I/O): 600
Run Time CPU0: 200
Run Time CPU1: 400
Total Simulation Time: 400
```

The lines have exactly the same meaning as before. A burst that was preempted on one processor and resumed on the other appears under *both* `CPU0` and `CPU1` with the same burst number, which is how migration is visible.

Trace. With `--trace` both programs additionally print a per-tick table in which each process is marked `RUN` (single processor) or `RUN0/RUN1` (dual processor, giving the processor it is running on), `RDY` (ready/waiting), `I/O` (blocked), `DONE` (terminated) or `-` (not yet arrived), together with the contents of the ready queue.

5 The Workloads

Five workload files are used for both parts. Their contents are summarised below. “Total CPU” is the sum of all CPU bursts of a process and therefore the minimum possible turnaround time for that process; “Total I/O” is the sum of its I/O bursts.

Workload 1 — *3 long CPU-bound processes, no I/O*

Process	Arrival	CPU bursts	I/O bursts	Total CPU	Total I/O
P1	0	200	—	200	0
P2	0	100	—	100	0
P3	10	300	—	300	0
Total				600	0

Workload 2 — *4 identical processes, long bursts with short I/O*

Process	Arrival	CPU bursts	I/O bursts	Total CPU	Total I/O
P1	0	50, 50, 50	5, 5	150	10
P2	0	50, 50, 50	5, 5	150	10
P3	0	50, 50, 50	5, 5	150	10
P4	0	50, 50, 50	5, 5	150	10
Total				600	40

Workload 3 — *4 identical processes, 1-unit CPU bursts*

Process	Arrival	CPU bursts	I/O bursts	Total CPU	Total I/O
P1	0	1, 1, 1	5, 5	3	10
P2	0	1, 1, 1	5, 5	3	10
P3	0	1, 1, 1	5, 5	3	10
P4	0	1, 1, 1	5, 5	3	10
Total				12	40

Workload 4 — *1 long, 1 medium and 1 short interactive process*

Process	Arrival	CPU bursts	I/O bursts	Total CPU	Total I/O
P1	0	30, 30	5	60	5
P2	10	10, 10, 10, 10, 10, 10	5, 5, 5, 5, 5	60	25
P3	15	3 ($\times 10$)	5 ($\times 9$)	30	45
Total				150	75

Workload 5 — *1 long, 1 medium and 4 short interactive processes*

Process	Arrival	CPU bursts	I/O bursts	Total CPU	Total I/O
P1	0	30, 30	5	60	5
P2	10	10, 10, 10, 10, 10, 10	5, 5, 5, 5, 5	60	25
P3	15	3 ($\times 10$)	5 ($\times 9$)	30	45
P4	15	3 ($\times 10$)	5 ($\times 9$)	30	45
P5	15	3 ($\times 10$)	5 ($\times 9$)	30	45
P6	15	3 ($\times 10$)	5 ($\times 9$)	30	45
Total				240	210

Part I

Single Processor

6 Detailed Turnaround Time Calculations

For every workload and every algorithm the tables below list, for each process, its arrival time A_i , its finish time F_i (the time unit at which its last CPU burst completes) and its turnaround time $TAT_i = F_i - A_i$, all measured on the **single processor**. The three summary rows give the average, maximum and minimum turnaround time over all processes of that workload. **MLFQ-NB** denotes MLFQ with the boost disabled and **MLFQ-WB** denotes MLFQ with a priority boost every 20 time units. Round Robin uses a time quantum of 2, matching the MLFQ quantum so that the comparison is fair.

6.1 Workload 1

Proc	A_i	FIFO			RR ($q = 2$)			MLFQ-NB			MLFQ-WB		
		F_i	$F_i - A_i$	TAT	F_i	$F_i - A_i$	TAT	F_i	$F_i - A_i$	TAT	F_i	$F_i - A_i$	TAT
P1	0	200	200 - 0	200	492	492 - 0	492	496	496 - 0	496	496	496 - 0	496
P2	0	300	300 - 0	300	294	294 - 0	294	298	298 - 0	298	298	298 - 0	298
P3	10	600	600 - 10	590	600	600 - 10	590	600	600 - 10	590	600	600 - 10	590
Average		363.33			458.67			461.33			461.33		
Maximum		590			590			590			590		
Minimum		200			294			298			298		

	FIFO	RR	MLFQ-NB	MLFQ-WB
Run time (CPU busy)	600	600	600	600
Total simulation time	600	600	600	600
CPU idle time	0	0	0	0
CPU utilisation	100.0%	100.0%	100.0%	100.0%
CPU segments (dispatches)	3	247	248	248

6.2 Workload 2

Proc	A_i	FIFO			RR ($q = 2$)			MLFQ-NB			MLFQ-WB		
		F_i	$F_i - A_i$	TAT	F_i	$F_i - A_i$	TAT	F_i	$F_i - A_i$	TAT	F_i	$F_i - A_i$	TAT
P1	0	450	450 - 0	450	588	588 - 0	588	588	588 - 0	588	594	594 - 0	594
P2	0	500	500 - 0	500	594	594 - 0	594	594	594 - 0	594	596	596 - 0	596
P3	0	550	550 - 0	550	598	598 - 0	598	598	598 - 0	598	598	598 - 0	598
P4	0	600	600 - 0	600	600	600 - 0	600	600	600 - 0	600	600	600 - 0	600
Average		525.00			595.00			595.00			597.00		
Maximum		600			600			600			600		
Minimum		450			588			588			594		

	FIFO	RR	MLFQ-NB	MLFQ-WB
Run time (CPU busy)	600	600	600	600
Total simulation time	600	600	600	600
CPU idle time	0	0	0	0
CPU utilisation	100.0%	100.0%	100.0%	100.0%
CPU segments (dispatches)	12	300	300	300

6.3 Workload 3

Proc	A_i	FIFO			RR ($q = 2$)			MLFQ-NB			MLFQ-WB		
		F_i	$F_i - A_i$	TAT	F_i	$F_i - A_i$	TAT	F_i	$F_i - A_i$	TAT	F_i	$F_i - A_i$	TAT
P1	0	13	13 - 0	13	13	13 - 0	13	13	13 - 0	13	13	13 - 0	13
P2	0	14	14 - 0	14	14	14 - 0	14	14	14 - 0	14	14	14 - 0	14
P3	0	15	15 - 0	15	15	15 - 0	15	15	15 - 0	15	15	15 - 0	15
P4	0	16	16 - 0	16	16	16 - 0	16	16	16 - 0	16	16	16 - 0	16
Average			14.50			14.50			14.50			14.50	
Maximum			16			16			16			16	
Minimum			13			13			13			13	

	FIFO	RR	MLFQ-NB	MLFQ-WB
Run time (CPU busy)	12	12	12	12
Total simulation time	16	16	16	16
CPU idle time	4	4	4	4
CPU utilisation	75.0%	75.0%	75.0%	75.0%
CPU segments (dispatches)	12	12	12	12

6.4 Workload 4

Proc	A_i	FIFO			RR ($q = 2$)			MLFQ-NB			MLFQ-WB		
		F_i	$F_i - A_i$	TAT	F_i	$F_i - A_i$	TAT	F_i	$F_i - A_i$	TAT	F_i	$F_i - A_i$	TAT
P1	0	73	73 - 0	73	126	126 - 0	126	135	135 - 0	135	136	136 - 0	136
P2	10	143	143 - 10	133	152	152 - 10	142	154	154 - 10	144	157	157 - 10	147
P3	15	178	178 - 15	163	141	141 - 15	126	142	142 - 15	127	127	127 - 15	112
Average			123.00			131.33			135.33			131.67	
Maximum			163			142			144			147	
Minimum			73			126			127			112	

	FIFO	RR	MLFQ-NB	MLFQ-WB
Run time (CPU busy)	150	150	150	150
Total simulation time	178	152	154	157
CPU idle time	28	2	4	7
CPU utilisation	84.3%	98.7%	97.4%	95.5%
CPU segments (dispatches)	18	68	67	62

6.5 Workload 5

Proc	A_i	FIFO			RR ($q = 2$)			MLFQ-NB			MLFQ-WB		
		F_i	$F_i - A_i$	TAT	F_i	$F_i - A_i$	TAT	F_i	$F_i - A_i$	TAT	F_i	$F_i - A_i$	TAT
P1	0	82	82 - 0	82	218	218 - 0	218	230	230 - 0	230	230	230 - 0	230
P2	10	180	180 - 10	170	244	244 - 10	234	243	243 - 10	233	243	243 - 10	233
P3	15	231	231 - 15	216	221	221 - 15	206	215	215 - 15	200	201	201 - 15	186
P4	15	234	234 - 15	219	224	224 - 15	209	218	218 - 15	203	202	202 - 15	187
P5	15	237	237 - 15	222	227	227 - 15	212	219	219 - 15	204	215	215 - 15	200
P6	15	240	240 - 15	225	230	230 - 15	215	222	222 - 15	207	216	216 - 15	201
Average			189.00			215.67			212.83			206.17	
Maximum			225			234			233			233	
Minimum			82			206			200			186	

	FIFO	RR	MLFQ-NB	MLFQ-WB
Run time (CPU busy)	240	240	240	240
Total simulation time	240	244	243	243
CPU idle time	0	4	3	3
CPU utilisation	100.0%	98.4%	98.8%	98.8%
CPU segments (dispatches)	48	131	131	131

7 Consolidated Summary

Workload	Algorithm	Avg TAT	Max TAT	Min TAT	Run time	Makespan	Util.
W1	FIFO	363.33	590	200	600	600	100.0%
	RR	458.67	590	294	600	600	100.0%
	MLFQ-NB	461.33	590	298	600	600	100.0%
	MLFQ-WB	461.33	590	298	600	600	100.0%
W2	FIFO	525.00	600	450	600	600	100.0%
	RR	595.00	600	588	600	600	100.0%
	MLFQ-NB	595.00	600	588	600	600	100.0%
	MLFQ-WB	597.00	600	594	600	600	100.0%
W3	FIFO	14.50	16	13	12	16	75.0%
	RR	14.50	16	13	12	16	75.0%
	MLFQ-NB	14.50	16	13	12	16	75.0%
	MLFQ-WB	14.50	16	13	12	16	75.0%
W4	FIFO	123.00	163	73	150	178	84.3%
	RR	131.33	142	126	150	152	98.7%
	MLFQ-NB	135.33	144	127	150	154	97.4%
	MLFQ-WB	131.67	147	112	150	157	95.5%
W5	FIFO	189.00	225	82	240	240	100.0%
	RR	215.67	234	206	240	244	98.4%
	MLFQ-NB	212.83	233	200	240	243	98.8%
	MLFQ-WB	206.17	233	186	240	243	98.8%

8 Comparative Graphs

In the bar charts below, W1–W5 denote workloads 1–5. Because the absolute turnaround times of the five workloads differ by more than an order of magnitude, the first two charts express each algorithm’s result as a percentage of the FIFO result for the same workload, so that all five workloads can be compared on one axis. A value above 100 % means the algorithm is *worse* than FIFO for that metric.

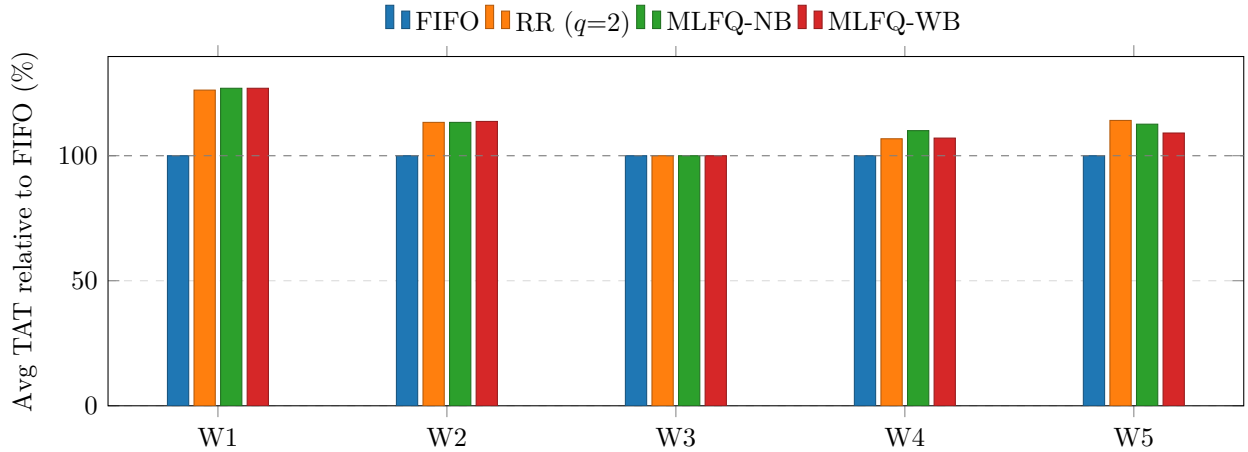


Figure 1: Average turnaround time of each algorithm as a percentage of the FIFO result for the same workload, single processor. FIFO is the 100 % baseline. No preemptive policy beats FIFO on any workload; on workload 3 all four are equal.

8.1 Per-process turnaround time

The aggregate averages hide how differently the policies treat *individual* processes. The following five charts break the turnaround time down per process, for every workload.

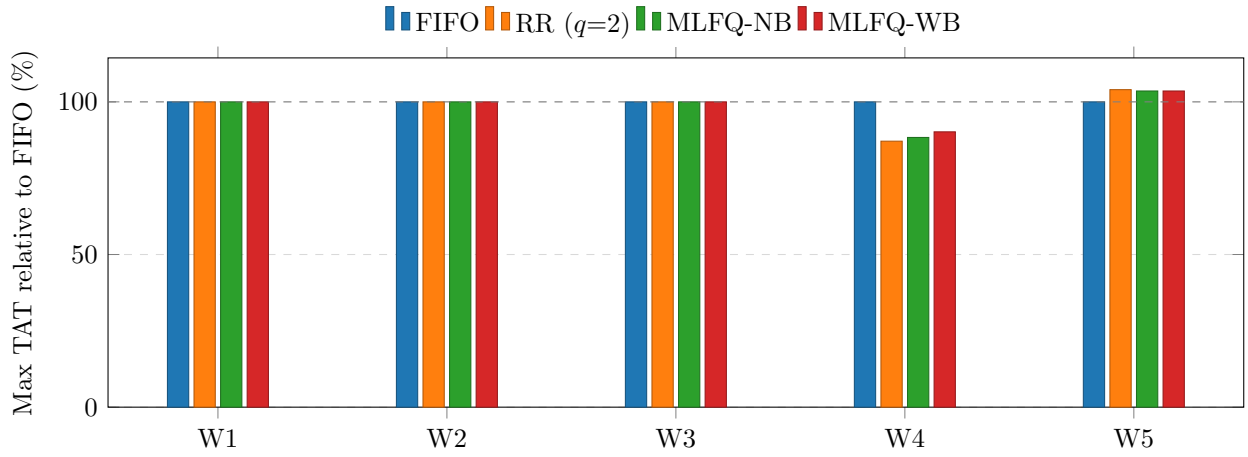


Figure 2: Maximum (worst-case) turnaround time relative to FIFO, single processor. On workload 4 every preemptive policy is clearly better than FIFO, which is where FIFO’s convoy effect hurts most.

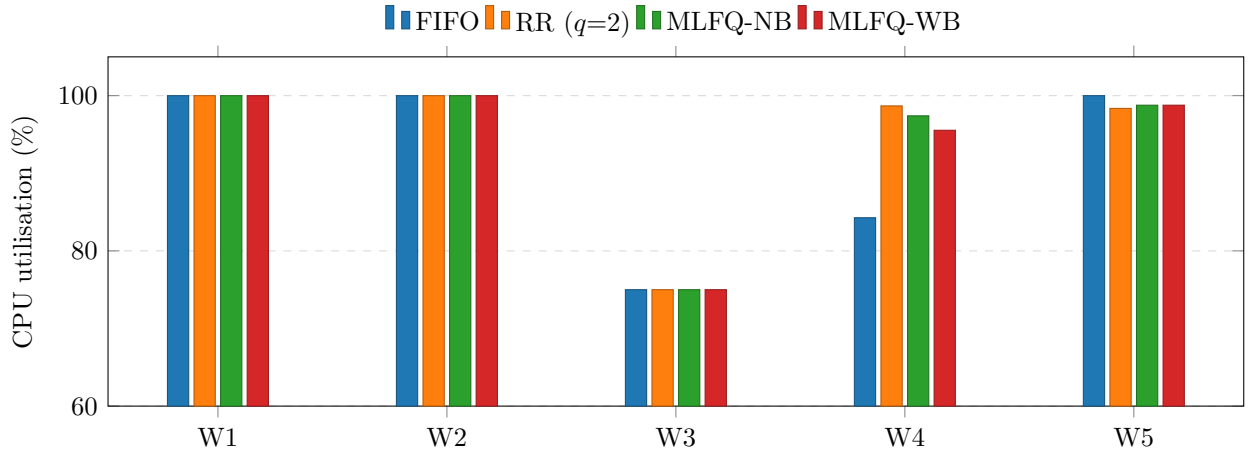


Figure 3: CPU utilisation, i.e. run time (excluding I/O) divided by total simulation time, single processor. The preemptive policies overlap I/O with computation far better on workload 4.

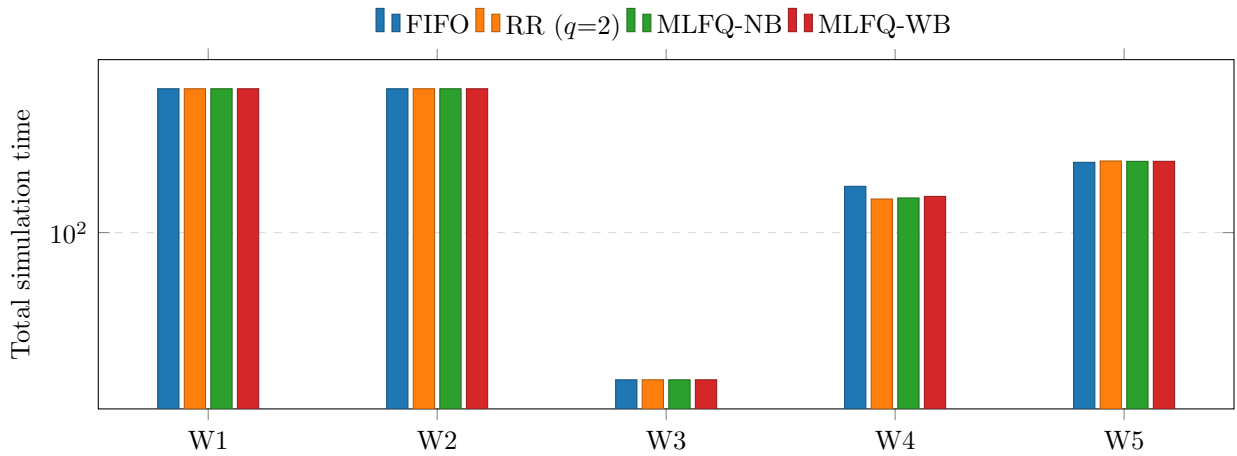


Figure 4: Makespan (total simulation time), single processor. Lower is better. The preemptive policies finish workload 4 substantially earlier than FIFO.

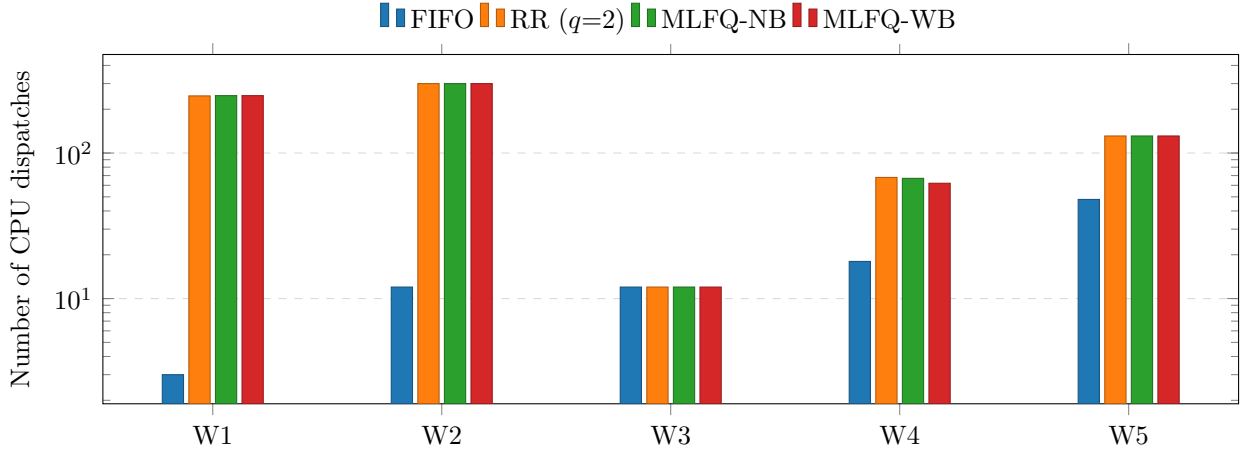


Figure 5: Number of contiguous CPU segments, a proxy for context-switch overhead, single processor. FIFO needs the fewest by a wide margin; this is the hidden cost of preemption.

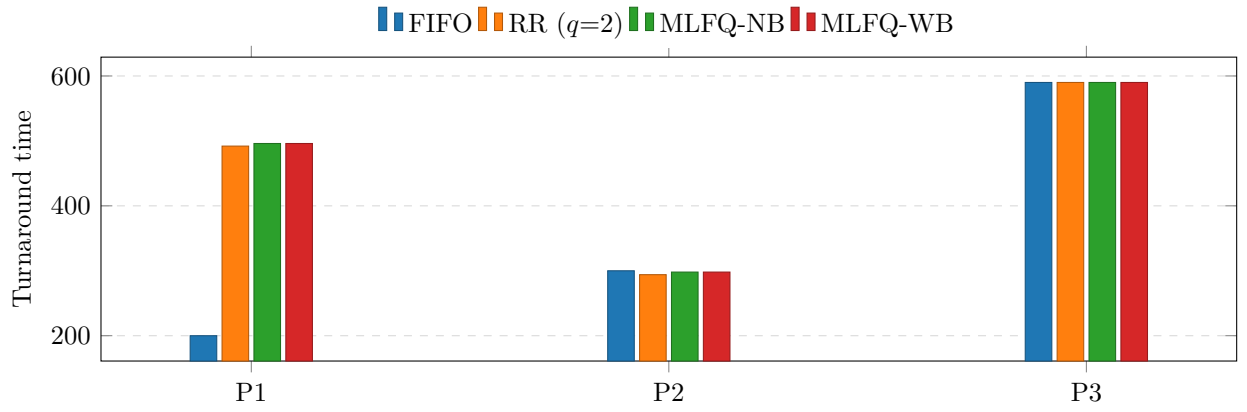


Figure 6: Per-process turnaround time, workload 1, single processor. Three pure CPU-bound processes and no I/O at all.

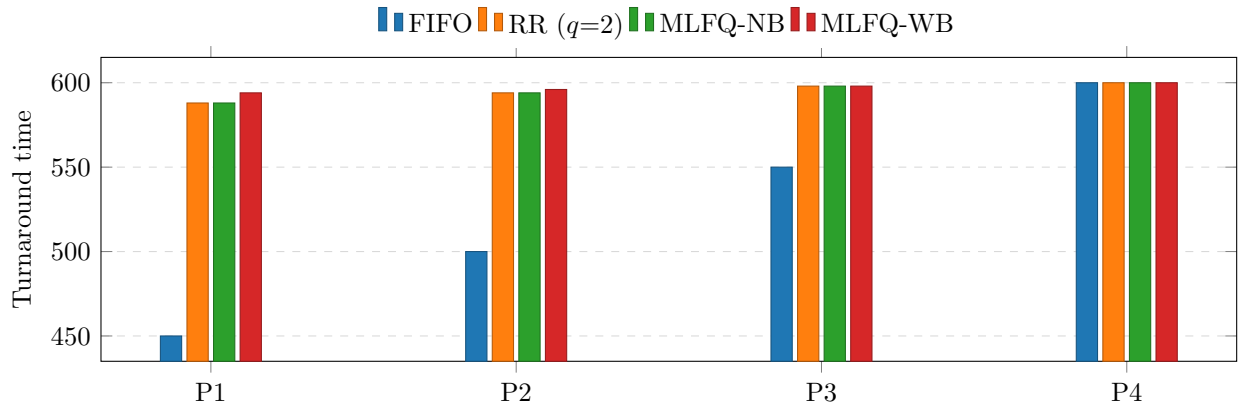


Figure 7: Per-process turnaround time, workload 2, single processor. Four identical processes with long (50-unit) CPU bursts.

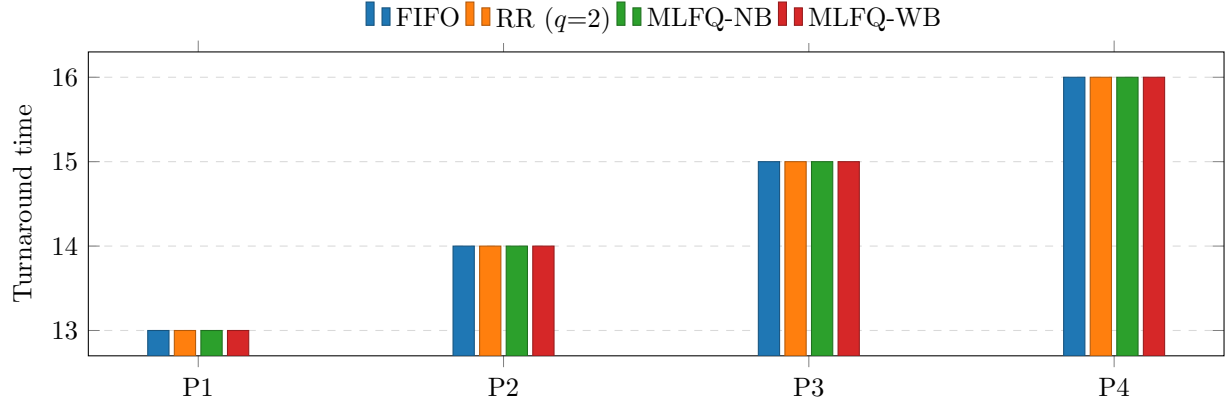


Figure 8: Per-process turnaround time, workload 3, single processor. Four identical processes whose CPU bursts are all 1 unit long.

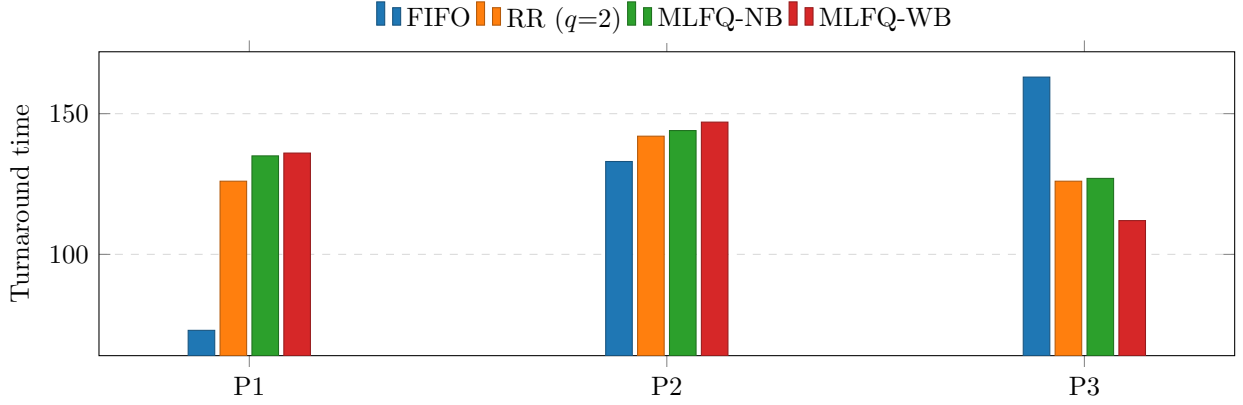


Figure 9: Per-process turnaround time, workload 4, single processor. P1 long CPU-bound, P2 medium, P3 short interactive.

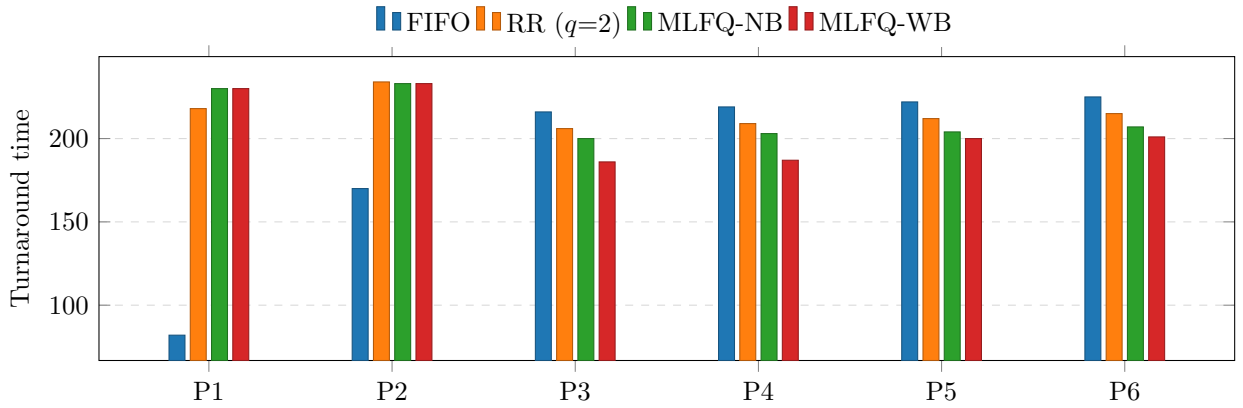


Figure 10: Per-process turnaround time, workload 5, single processor. P1 long, P2 medium, P3-P6 short interactive, all arriving at $t = 15$.

8.2 Schedule visualisation (Gantt charts)

The clearest way to see *why* the numbers differ is to look at the schedules themselves. In the charts below each row is a process, each coloured block is a contiguous stretch of CPU time, and white space in a row means the process is either blocked on I/O or waiting in a ready queue. White space spanning *all* rows of a chart is an idle CPU.

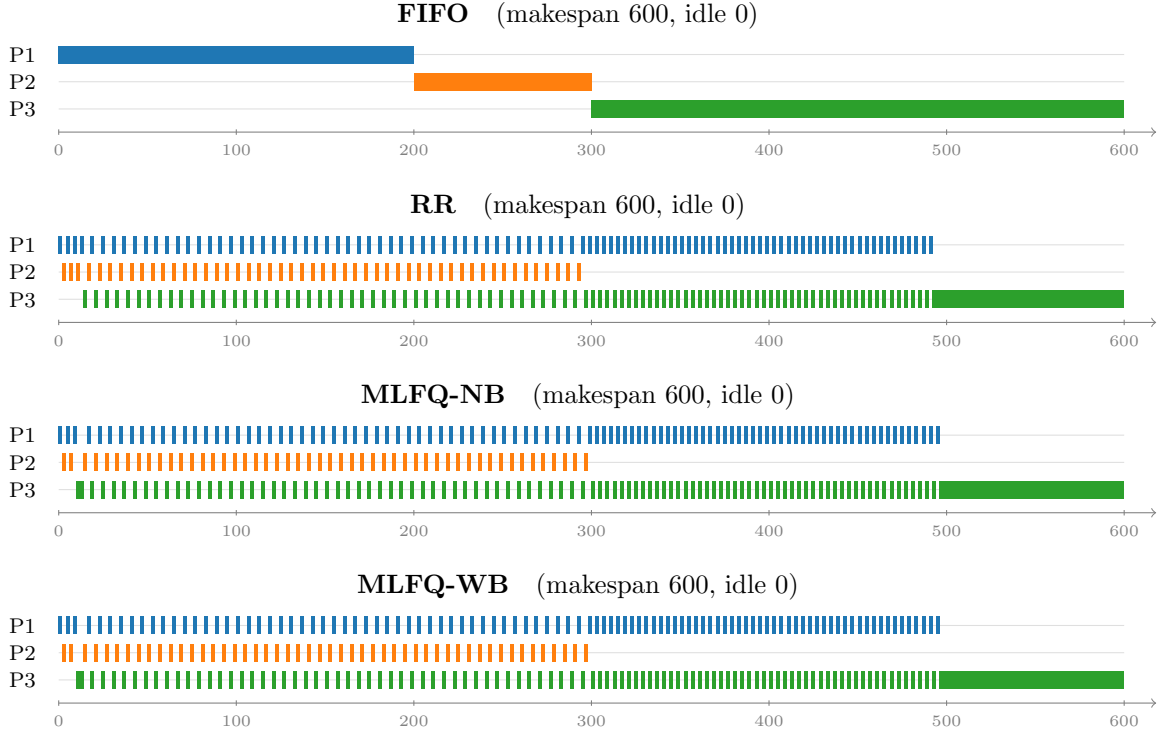


Figure 11: Workload 1 (no I/O), single processor. FIFO runs each process to completion in turn — three solid blocks. The preemptive policies shred the same work into hundreds of slices; the makespan is identical (600) because the CPU never idles, but P1 finishes far later, which is why their average turnaround is worse.

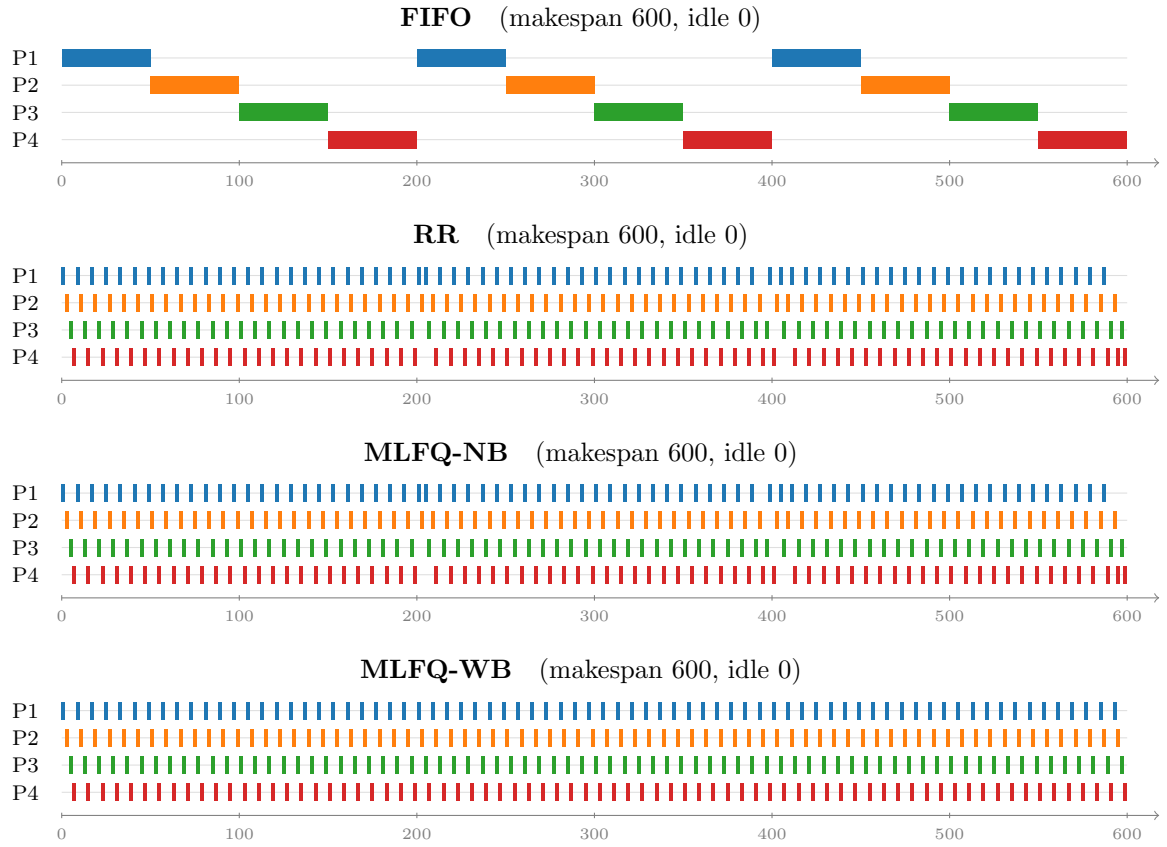


Figure 12: Workload 2, single processor. The short 5-unit I/O bursts are almost entirely hidden behind other processes' computation, so all four policies keep the CPU busy for the full 600 units.

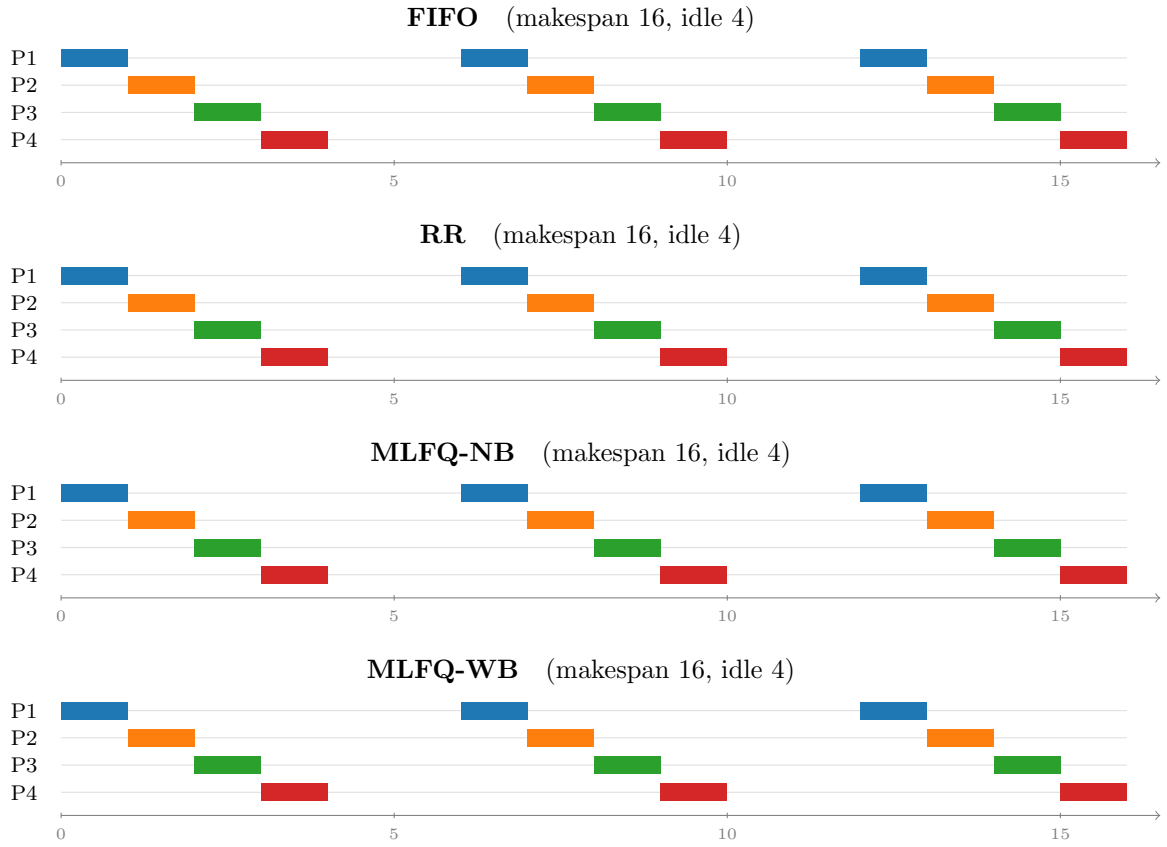


Figure 13: Workload 3, single processor. Every CPU burst is 1 unit, shorter than the quantum of 2, so preemption never triggers and all four schedules are identical. The CPU is idle in 2 gaps of 2 units (4 units in total), when every process is simultaneously blocked on I/O.

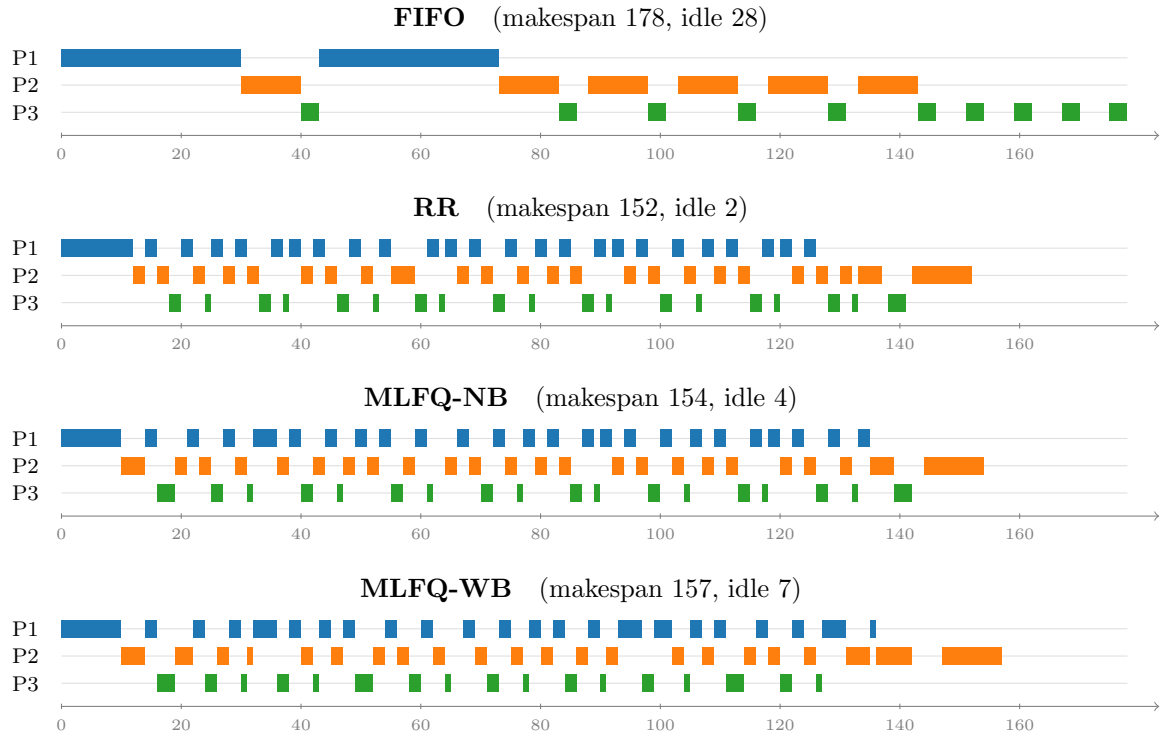


Figure 14: Workload 4, single processor. FIFO (top) leaves the CPU idle for 28 time units and stretches P3's completion to 178; the preemptive policies interleave the three processes, cut idle time to 2–7 units and finish 21–26 time units earlier.

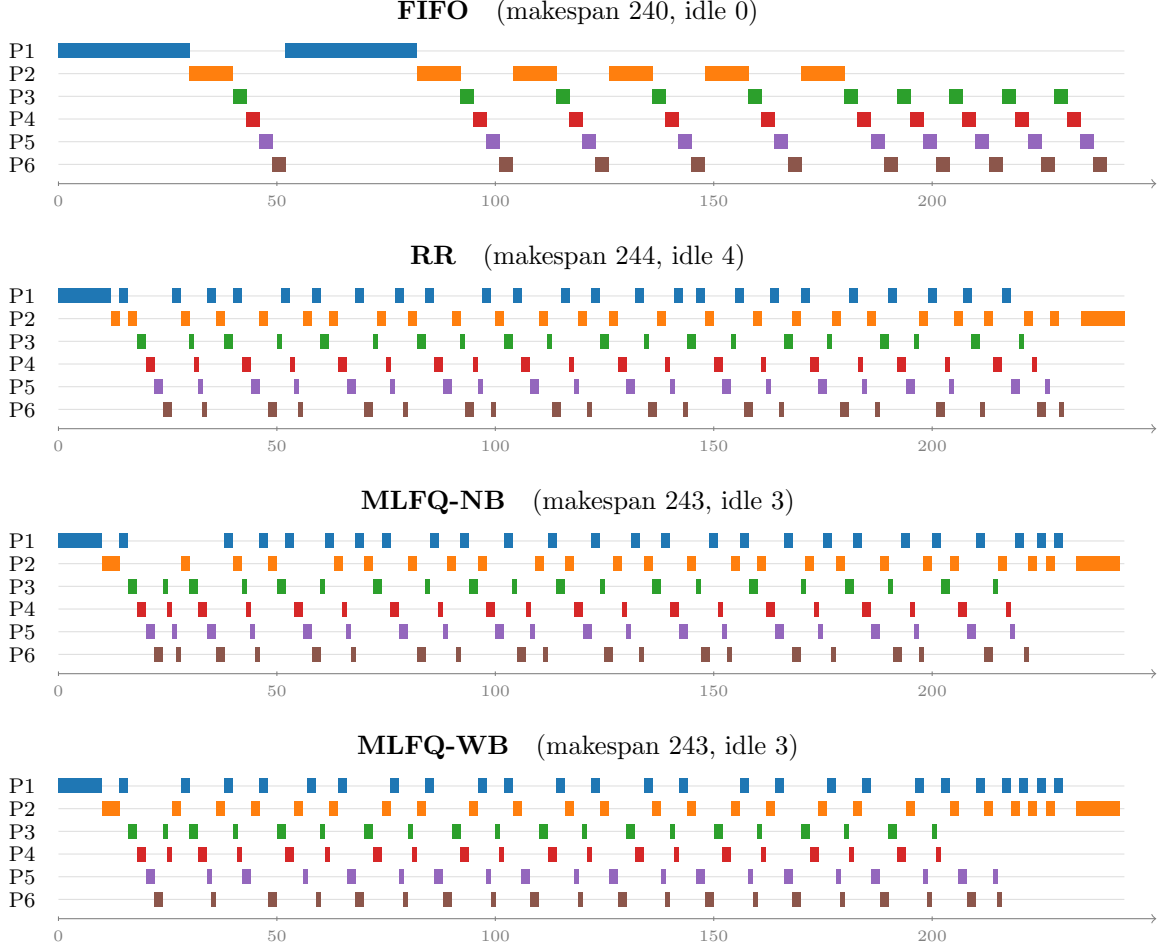


Figure 15: Workload 5, single processor. The four short interactive processes P3–P6 all arrive at $t = 15$. Under FIFO they queue behind P1 and P2; the preemptive policies interleave them immediately.

9 Additional Experiments

9.1 Effect of the Round Robin time quantum

The assignment asks for Round Robin to be studied with different values of the time quantum. The table reports the average turnaround time on the single processor for $q \in \{1, 2, 4, 8, 16\}$ on all five workloads.

Workload	$q = 1$	$q = 2$	$q = 4$	$q = 8$	$q = 16$
Workload 1	459.33	458.67	458.00	458.00	463.33
Workload 2	598.50	595.00	596.00	597.00	597.00
Workload 3	14.50	14.50	14.50	14.50	14.50
Workload 4	130.00	131.33	131.67	128.33	129.00
Workload 5	203.83	215.67	207.17	205.67	196.33

The best quantum for each workload is shown in bold. Because the absolute turnaround times differ by more than an order of magnitude between workloads, the graph below normalises each workload against its own best quantum, so that all five curves can be compared on a single set of axes.

9.2 Effect of the MLFQ priority boost interval

The assignment asks for a no-boost experiment to be compared against a boost every 20 time units. To place that comparison in context, the boost interval B was swept over $\{0, 5, 10, 20, 40, 80\}$ on

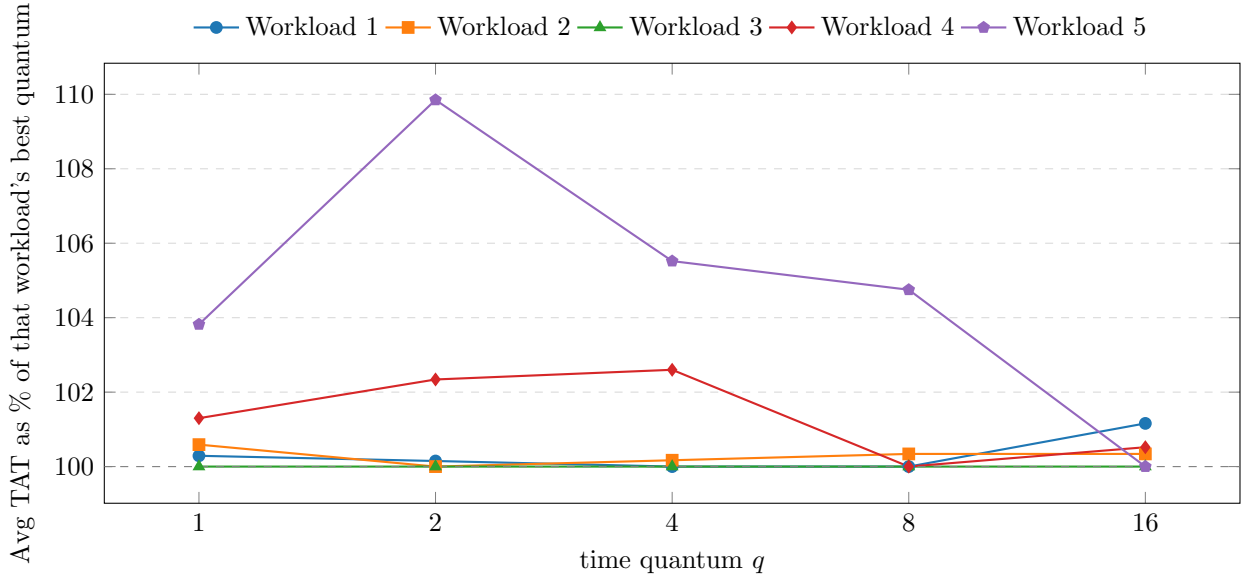


Figure 16: Average turnaround time versus Round Robin time quantum on a single processor, all five workloads on one axis. Each curve is normalised against the best quantum for that workload (100 % = best). Workload 3 is perfectly flat because its 1-unit CPU bursts never reach any quantum; the others respond non-monotonically, showing that the quantum interacts with the specific burst lengths of the workload.

the single processor, where $B = 0$ means boosting is disabled.

Workload	$B = 0$ (off)	$B = 5$	$B = 10$	$B = 20$	$B = 40$	$B = 80$
Workload 1	461.33	449.67	458.67	461.33	461.33	461.33
Workload 2	595.00	597.50	597.00	597.00	597.00	596.00
Workload 3	14.50	14.50	14.50	14.50	14.50	14.50
Workload 4	135.33	124.67	124.33	131.67	136.67	135.33
Workload 5	212.83	208.00	207.17	206.17	198.17	202.17

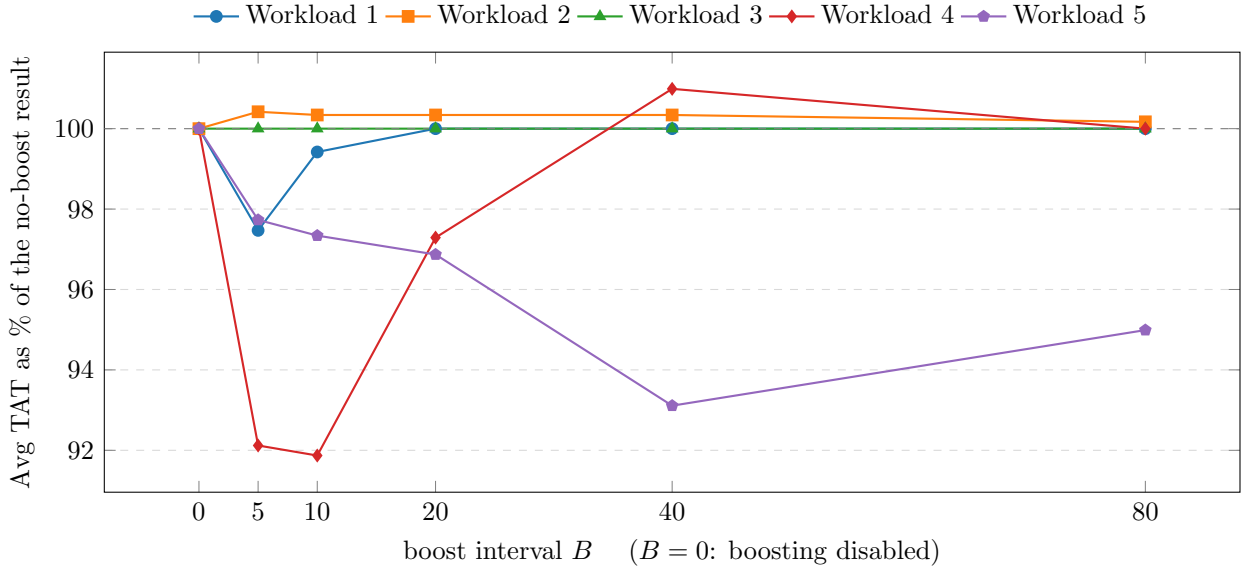


Figure 17: Average turnaround time versus MLFQ priority boost interval on a single processor, all five workloads on one axis. Each curve is normalised against that workload’s own no-boost result ($B = 0$), so the dashed 100% line is “no boost at all” and any point *below* it means boosting improved the average turnaround time. Workload 3 is flat because no demotion ever occurs, so a boost has nothing to undo.

10 Findings and Analysis

10.1 Finding 1: FIFO has the lowest *average* turnaround time

On a single processor FIFO produced the lowest average turnaround time on four workloads and tied with every other policy on workload 3:

Workload	FIFO	RR	MLFQ-NB	MLFQ-WB
Workload 1	363.33	458.67	461.33	461.33
Workload 2	525.00	595.00	595.00	597.00
Workload 3	14.50	14.50	14.50	14.50
Workload 4	123.00	131.33	135.33	131.67
Workload 5	189.00	215.67	212.83	206.17

This is not an accident and it is not a defect of the preemptive policies. Average turnaround time rewards letting *some* processes finish as early as possible. FIFO runs one process to the end of its burst before starting another, so early processes complete early. Round Robin and MLFQ interleave every runnable process, so they all make progress together and consequently tend to finish at similar, later times.

Workload 1 shows this in its purest form. It contains three pure CPU-bound processes (200, 100 and 300 units) with no I/O at all, so the total work is 600 units and *every* policy has a makespan of exactly 600 — the CPU is never idle and nothing can change the finishing time of the last process. The policies differ only in *when the first two* processes finish:

Policy	P1 finishes	P2 finishes	P3 finishes
FIFO	200	300	600
RR	492	294	600
MLFQ-NB	496	298	600
MLFQ-WB	496	298	600

Under FIFO, P1 finishes at time 200 and P2 at 300. Under Round Robin, P1 is dragged out to 492 — 292 time units later — while P2 finishes at 294, only 6 time units *earlier*. P3 finishes at 600

in both cases. Time-slicing therefore made one process much slower in exchange for a tiny gain for another, which is exactly why the average rises from 363.33 to 458.67.

10.2 Finding 2: FIFO loses badly on *worst-case* turnaround time

Average turnaround time alone is a misleading summary. On workload 4 the ranking reverses when the maximum is considered:

Workload 4	FIFO	RR	MLFQ-NB	MLFQ-WB
Average TAT	123.00	131.33	135.33	131.67
Maximum TAT	163	142	144	147
Minimum TAT	73	126	127	112

FIFO’s worst-case turnaround is 163, versus 142 for Round Robin — and its spread between the best- and worst-served process is 73 to 163, a range of 90 time units. Round Robin’s spread is only 126 to 142, a range of 16. This is the **convoy effect**: workload 4’s P1 is a 30-unit CPU-bound process, and under FIFO the 3-unit interactive process P3 must wait behind it every single time. FIFO achieves a good average by being extremely unfair.

If the goal is an interactive system, the maximum and the spread matter far more than the mean, and on that criterion every preemptive policy beats FIFO here.

10.3 Finding 3: preemption dramatically improves CPU utilisation

The most practically important result concerns how well each policy overlaps I/O with computation:

Workload 4	FIFO	RR	MLFQ-NB	MLFQ-WB
Run time (CPU busy)	150	150	150	150
Makespan	178	152	154	157
Idle time	28	2	4	7
Utilisation	84.3%	98.7%	97.4%	95.5%

All four policies must execute the same 150 units of CPU work, so run time is identical. What differs is how much time is *wasted*. FIFO idles the CPU for 28 time units (84.3 % utilisation) because it lets each process run its full burst, which delays the moment the other processes issue their I/O requests; eventually all three are blocked simultaneously and the CPU has nothing to do. Round Robin idles for only 2 units (98.7%), because slicing forces every process to reach its I/O request early, so I/O requests are spread out and overlap with computation.

The consequence is that Round Robin finishes the entire workload at time 152 instead of FIFO’s 178 — **26 time units, or 15 %, sooner**. On a real machine this is the difference between a saturated CPU and one sitting idle waiting for disks.

10.4 Finding 4: the priority boost trades worst case for average

Comparing MLFQ with and without the boost:

	Workload 4		Workload 5	
	No boost	Boost 20	No boost	Boost 20
Average TAT	135.33	131.67	212.83	206.17
Maximum TAT	144	147	233	233
Minimum TAT	127	112	200	186

On both mixed workloads the boost *improves* the average turnaround time (135.33 → 131.67 on workload 4; 212.83 → 206.17 on workload 5). On workload 4 it simultaneously makes the *maximum* slightly worse (144 → 147).

The mechanism is visible in the minimum turnaround column: on workload 4 the boost improves the best-served process from 127 to 112. Boosting periodically lifts processes that have sunk to Q_2 back into Q_0 , letting them make a burst of progress they would otherwise have made much later. That helps those processes (and the average), while the short processes occasionally have to queue behind a freshly promoted long process, which costs them a little in the worst case.

This is precisely the trade-off the boost is designed to make: it sacrifices the scheduler’s accumulated knowledge of which processes are CPU-bound, in exchange for a hard guarantee that no process waits more than B time units at low priority.

10.5 Finding 5: neither parameter is monotonic

Neither the Round Robin quantum nor the boost interval has a simple “bigger is better” relationship with turnaround time. On workload 5 the average turnaround time under Round Robin is 203.83, 215.67, 207.17, 205.67 and 196.33 for $q = 1, 2, 4, 8, 16$ respectively — it gets worse from $q = 1$ to $q = 2$, then improves steadily. On workload 4 the best boost interval was $B = 10$ rather than the assignment’s $B = 20$, and $B = 40$ (136.67) was worse than no boost at all (135.33).

The reason is that these parameters interact with the specific burst lengths of the workload. A quantum that happens to align with a common burst length causes many processes to finish exactly at a quantum boundary, changing the queue order for everything that follows. There is therefore no universally optimal quantum; it must be tuned to the expected workload.

10.6 Finding 6: when all four algorithms coincide

Workload 3 produced *identical* results for all four policies: average 14.50, maximum 16, minimum 13, makespan 16. Every CPU burst in workload 3 is exactly 1 time unit long, and both Round Robin and MLFQ use a quantum of 2. A burst therefore always completes before the quantum can expire, so preemption never fires and no MLFQ demotion ever occurs. With no demotions, every process stays in Q_0 forever and the boost has nothing to do. All three policies degenerate to plain FIFO.

This is a useful sanity check on the implementation, and it illustrates a real principle: a scheduling policy can only differentiate itself when the quantum is small relative to the burst lengths.

10.7 Finding 7: the hidden cost of preemption

Counting the number of contiguous CPU segments gives a proxy for context-switch overhead:

Workload	FIFO	RR	MLFQ-NB	MLFQ-WB
Workload 1	3	247	248	248
Workload 2	12	300	300	300
Workload 3	12	12	12	12
Workload 4	18	68	67	62
Workload 5	48	131	131	131

On workload 1 FIFO needs 3 dispatches while Round Robin needs 247 — over 82 times more. The simulator models context switches as free, so this cost does not appear in any of the turnaround numbers. On real hardware each switch costs pipeline flushes, TLB and cache pollution, and scheduler bookkeeping. A complete evaluation would have to weigh Round Robin’s better responsiveness against this overhead, and it is the main practical reason why real systems do not use a quantum of 1.

11 Summary of Part I

Policy	Strengths	Weaknesses	Best suited to
FIFO	Lowest average turnaround on every workload (tied on W3); minimal context-switch overhead; trivial to implement; no starvation	Severe convoy effect; worst maximum turnaround (163 on W4); poor CPU utilisation (84.3 % on W4); no responsiveness	Batch systems where throughput matters and latency does not
Round Robin	Bounded waiting; excellent fairness (W4 spread only 16 units); best CPU utilisation (98.7 % on W4); shortest makespan on W4	Worse average turnaround than FIFO on every workload except W3; very high switch count; performance sensitive to q ; cannot distinguish process types	Time-sharing systems with similar processes
MLFQ (no boost)	Infers process behaviour with no prior knowledge; favours interactive jobs; good utilisation (97.4 % on W4)	CPU-bound jobs can starve in Q_2 ; can be gamed by yielding before quantum expiry	General-purpose systems with mixed workloads
MLFQ (boost 20)	Removes starvation entirely; improved average over no-boost on both mixed workloads; adapts to changing behaviour	Discards learned priority information; slightly worse maximum on W4; B needs tuning	Interactive general-purpose operating systems

On a single processor, no algorithm is best on every metric. FIFO wins on average turnaround time; the preemptive policies win on worst-case turnaround, fairness and CPU utilisation. MLFQ recovers most of Round Robin’s fairness while automatically identifying interactive processes, and the boost removes its starvation weakness at a small cost in the worst case.

Part II

Dual Processor

12 Overview of Part II

Part II repeats every experiment of Part I on the dual-processor simulator described in Section 2.2: the same five workloads, the same four policies, the same quantum of 2 and boost interval of 20, and the same parameter sweeps. Only the number of processors has changed. The structure of this part mirrors Part I exactly; a direct side-by-side comparison of the two follows in Part III.

13 Detailed Turnaround Time Calculations

For every workload and every algorithm the tables below list, for each process, its arrival time A_i , its finish time F_i (the time unit at which its last CPU burst completes) and its turnaround time $TAT_i = F_i - A_i$, all measured on the **dual processor**. The three summary rows give the average, maximum and minimum turnaround time over all processes of that workload. **MLFQ-NB** denotes MLFQ with the boost disabled and **MLFQ-WB** denotes MLFQ with a priority boost every 20 time units. Round Robin uses a time quantum of 2, matching the MLFQ quantum so that the comparison is fair.

A second table under each workload gives the processor-level figures: the busy time of each CPU, the idle time summed over both CPUs, the utilisation of the pair of CPUs, and the number of CPU bursts that *migrated*, i.e. were started on one processor and finished on the other.

13.1 Workload 1

Proc	A_i	FIFO			RR ($q = 2$)			MLFQ-NB			MLFQ-WB		
		F_i	$F_i - A_i$	TAT	F_i	$F_i - A_i$	TAT	F_i	$F_i - A_i$	TAT	F_i	$F_i - A_i$	TAT
P1	0	200	200 - 0	200	244	244 - 0	244	246	246 - 0	246	246	246 - 0	246
P2	0	100	100 - 0	100	144	144 - 0	144	146	146 - 0	146	146	146 - 0	146
P3	10	400	400 - 10	390	356	356 - 10	346	354	354 - 10	344	354	354 - 10	344
Average		230.00			244.67			245.33			245.33		
Maximum		390			346			344			344		
Minimum		100			144			146			146		

	FIFO	RR	MLFQ-NB	MLFQ-WB
Run time CPU0	200	356	354	354
Run time CPU1	400	244	246	246
Run time, both CPUs	600	600	600	600
Total simulation time	400	356	354	354
Idle time, both CPUs	200	112	108	108
Utilisation of both CPUs	75.0%	84.3%	84.7%	84.7%
CPU segments (dispatches)	3	136	140	140
Bursts migrated between CPUs	0 of 3	3 of 3	3 of 3	3 of 3

13.2 Workload 2

Proc	A_i	FIFO			RR ($q = 2$)			MLFQ-NB			MLFQ-WB		
		F_i	$F_i - A_i$	TAT	F_i	$F_i - A_i$	TAT	F_i	$F_i - A_i$	TAT	F_i	$F_i - A_i$	TAT
P1	0	250	250 - 0	250	304	304 - 0	304	304	304 - 0	304	304	304 - 0	304
P2	0	250	250 - 0	250	304	304 - 0	304	304	304 - 0	304	304	304 - 0	304
P3	0	300	300 - 0	300	306	306 - 0	306	306	306 - 0	306	306	306 - 0	306
P4	0	300	300 - 0	300	306	306 - 0	306	306	306 - 0	306	306	306 - 0	306
Average			275.00			305.00			305.00			305.00	
Maximum			300			306			306			306	
Minimum			250			304			304			304	

	FIFO	RR	MLFQ-NB	MLFQ-WB
Run time CPU0	300	300	300	300
Run time CPU1	300	300	300	300
Run time, both CPUs	600	600	600	600
Total simulation time	300	306	306	306
Idle time, both CPUs	0	12	12	12
Utilisation of both CPUs	100.0%	98.0%	98.0%	98.0%
CPU segments (dispatches)	12	300	300	300
Bursts migrated between CPUs	0 of 12	0 of 12	0 of 12	0 of 12

13.3 Workload 3

Proc	A_i	FIFO			RR ($q = 2$)			MLFQ-NB			MLFQ-WB		
		F_i	$F_i - A_i$	TAT	F_i	$F_i - A_i$	TAT	F_i	$F_i - A_i$	TAT	F_i	$F_i - A_i$	TAT
P1	0	13	13 - 0	13	13	13 - 0	13	13	13 - 0	13	13	13 - 0	13
P2	0	13	13 - 0	13	13	13 - 0	13	13	13 - 0	13	13	13 - 0	13
P3	0	14	14 - 0	14	14	14 - 0	14	14	14 - 0	14	14	14 - 0	14
P4	0	14	14 - 0	14	14	14 - 0	14	14	14 - 0	14	14	14 - 0	14
Average			13.50			13.50			13.50			13.50	
Maximum			14			14			14			14	
Minimum			13			13			13			13	

	FIFO	RR	MLFQ-NB	MLFQ-WB
Run time CPU0	6	6	6	6
Run time CPU1	6	6	6	6
Run time, both CPUs	12	12	12	12
Total simulation time	14	14	14	14
Idle time, both CPUs	16	16	16	16
Utilisation of both CPUs	42.9%	42.9%	42.9%	42.9%
CPU segments (dispatches)	12	12	12	12
Bursts migrated between CPUs	0 of 12	0 of 12	0 of 12	0 of 12

13.4 Workload 4

Proc	A_i	FIFO			RR ($q = 2$)			MLFQ-NB			MLFQ-WB		
		F_i	$F_i - A_i$	TAT	F_i	$F_i - A_i$	TAT	F_i	$F_i - A_i$	TAT	F_i	$F_i - A_i$	TAT
P1	0	65	65 - 0	65	69	69 - 0	69	72	72 - 0	72	73	73 - 0	73
P2	10	96	96 - 10	86	101	101 - 10	91	101	101 - 10	91	100	100 - 10	90
P3	15	108	108 - 15	93	94	94 - 15	79	94	94 - 15	79	93	93 - 15	78
Average			81.33			79.67			80.67			80.33	
Maximum			93			91			91			90	
Minimum			65			69			72			73	

	FIFO	RR	MLFQ-NB	MLFQ-WB
Run time CPU0	95	90	90	91
Run time CPU1	55	60	60	59
Run time, both CPUs	150	150	150	150
Total simulation time	108	101	101	100
Idle time, both CPUs	66	52	52	50
Utilisation of both CPUs	69.4%	74.3%	74.3%	75.0%
CPU segments (dispatches)	18	40	44	45
Bursts migrated between CPUs	0 of 18	11 of 18	12 of 18	11 of 18

13.5 Workload 5

Proc	A_i	FIFO			RR ($q = 2$)			MLFQ-NB			MLFQ-WB		
		F_i	$F_i - A_i$	TAT	F_i	$F_i - A_i$	TAT	F_i	$F_i - A_i$	TAT	F_i	$F_i - A_i$	TAT
P1	0	70	70 - 0	70	103	103 - 0	103	106	106 - 0	106	112	112 - 0	112
P2	10	111	111 - 10	101	130	130 - 10	120	133	133 - 10	123	135	135 - 10	125
P3	15	125	125 - 15	110	117	117 - 15	102	115	115 - 15	100	120	120 - 15	105
P4	15	128	128 - 15	113	119	119 - 15	104	118	118 - 15	103	121	121 - 15	106
P5	15	130	130 - 15	115	121	121 - 15	106	121	121 - 15	106	121	121 - 15	106
P6	15	131	131 - 15	116	124	124 - 15	109	122	122 - 15	107	122	122 - 15	107
Average		104.17			107.33			107.50			110.17		
Maximum		116			120			123			125		
Minimum		70			102			100			105		

	FIFO	RR	MLFQ-NB	MLFQ-WB
Run time CPU0	129	130	132	131
Run time CPU1	111	110	108	109
Run time, both CPUs	240	240	240	240
Total simulation time	131	130	133	135
Idle time, both CPUs	22	20	26	30
Utilisation of both CPUs	91.6%	92.3%	90.2%	88.9%
CPU segments (dispatches)	48	123	120	113
Bursts migrated between CPUs	0 of 48	27 of 48	17 of 48	11 of 48

14 Consolidated Summary

Workload	Algorithm	Avg TAT	Max TAT	Min TAT	Run time	Makespan	Util.
W1	FIFO	230.00	390	100	600	400	75.0%
	RR	244.67	346	144	600	356	84.3%
	MLFQ-NB	245.33	344	146	600	354	84.7%
	MLFQ-WB	245.33	344	146	600	354	84.7%
W2	FIFO	275.00	300	250	600	300	100.0%
	RR	305.00	306	304	600	306	98.0%
	MLFQ-NB	305.00	306	304	600	306	98.0%
	MLFQ-WB	305.00	306	304	600	306	98.0%
W3	FIFO	13.50	14	13	12	14	42.9%
	RR	13.50	14	13	12	14	42.9%
	MLFQ-NB	13.50	14	13	12	14	42.9%
	MLFQ-WB	13.50	14	13	12	14	42.9%
W4	FIFO	81.33	93	65	150	108	69.4%
	RR	79.67	91	69	150	101	74.3%
	MLFQ-NB	80.67	91	72	150	101	74.3%
	MLFQ-WB	80.33	90	73	150	100	75.0%
W5	FIFO	104.17	116	70	240	131	91.6%
	RR	107.33	120	102	240	130	92.3%
	MLFQ-NB	107.50	123	100	240	133	90.2%
	MLFQ-WB	110.17	125	105	240	135	88.9%

Run time is summed over both processors, and utilisation is run time divided by $2 \times$ makespan.

15 Comparative Graphs

In the bar charts below, W1–W5 denote workloads 1–5. Because the absolute turnaround times of the five workloads differ by more than an order of magnitude, the first two charts express each algorithm’s result as a percentage of the FIFO result for the same workload, so that all five workloads can be compared on one axis. A value above 100 % means the algorithm is *worse* than FIFO for that metric.

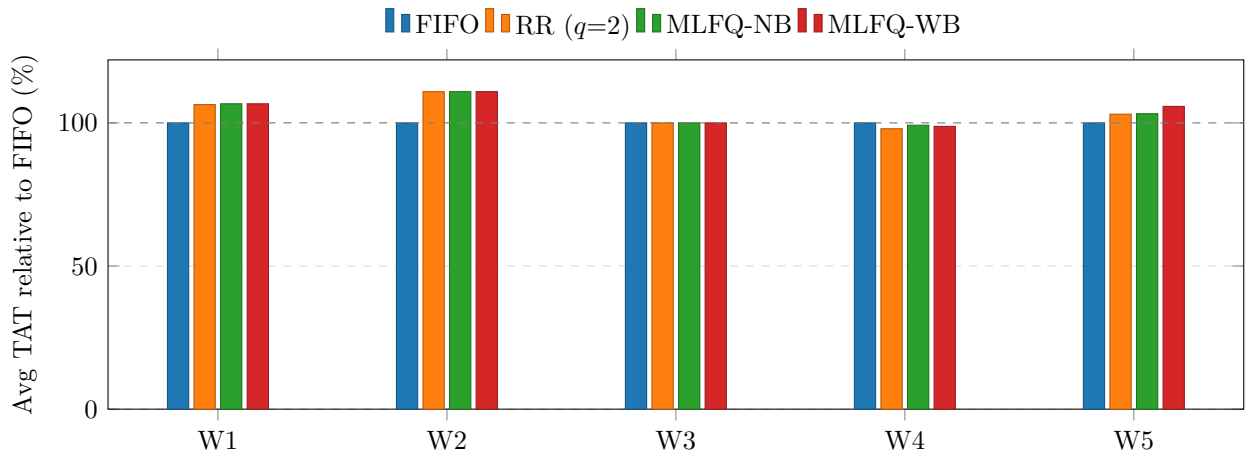


Figure 18: Average turnaround time of each algorithm as a percentage of the FIFO result for the same workload, dual processor. Unlike on one processor, FIFO is no longer best everywhere: on workload 4 all three preemptive policies fall below the 100 % line.

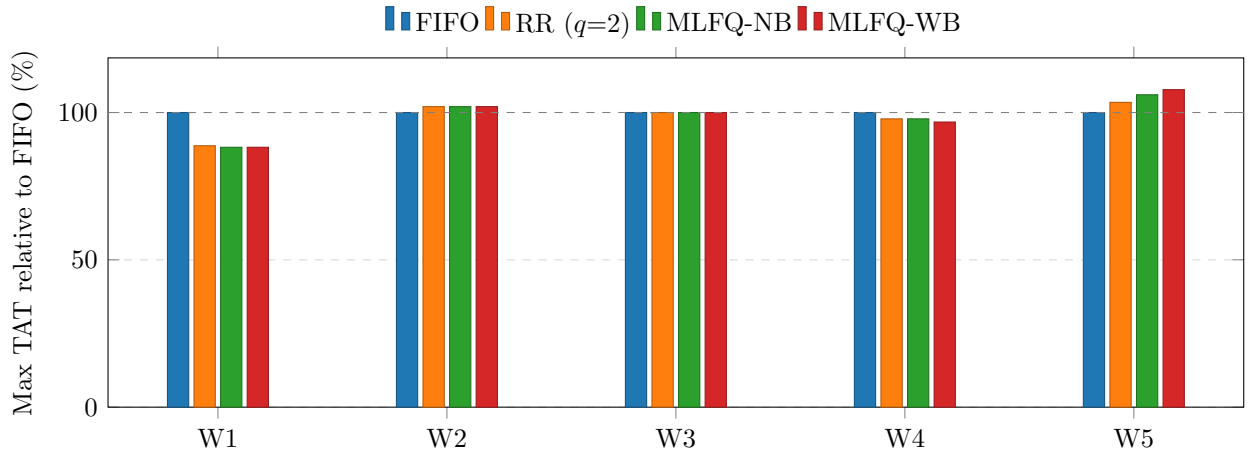


Figure 19: Maximum (worst-case) turnaround time relative to FIFO, dual processor. The preemptive policies have a lower worst case than FIFO on W1 and W4, and a higher one on W2 and W5.

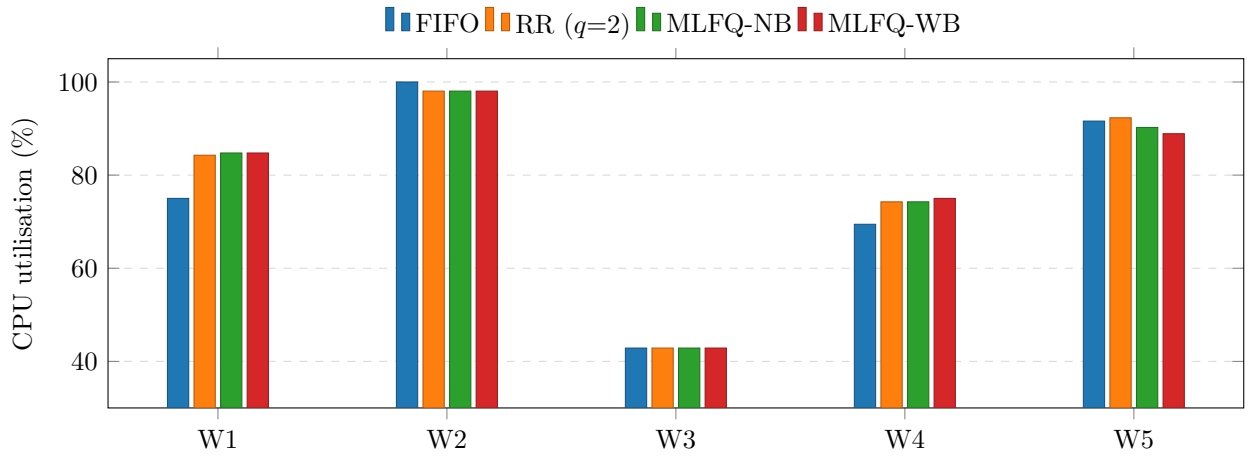


Figure 20: Utilisation of the pair of processors, i.e. total run time divided by $2 \times$ makespan. Workload 3 keeps the processors busy less than half the time, because it is limited by I/O rather than by processor capacity.

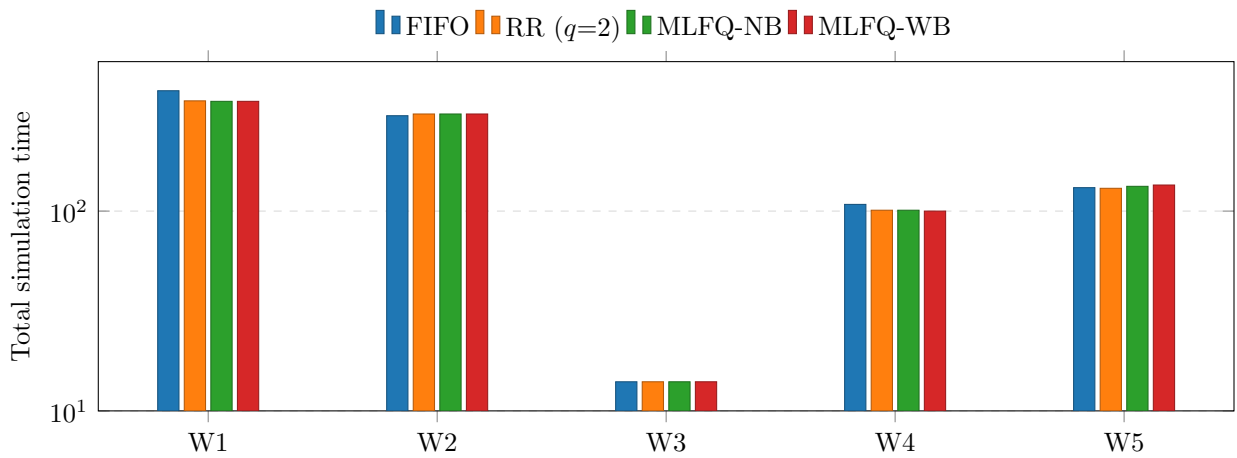


Figure 21: Makespan (total simulation time), dual processor. Lower is better. FIFO has the longest makespan on W1 and W4.

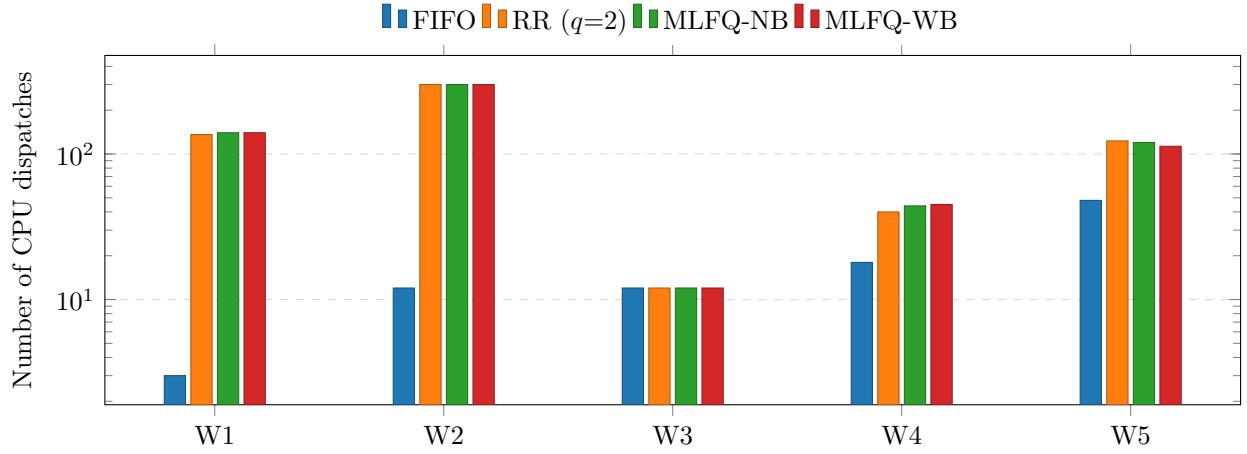


Figure 22: Number of contiguous CPU segments over both processors, dual processor. FIFO again needs the fewest on every workload.

15.1 Per-process turnaround time

The aggregate averages hide how differently the policies treat *individual* processes. The following five charts break the turnaround time down per process, for every workload.

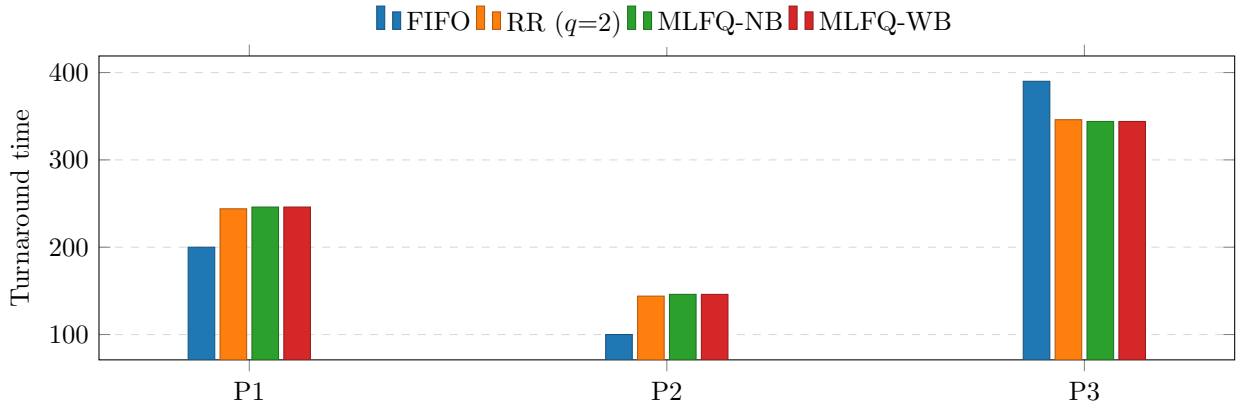


Figure 23: Per-process turnaround time, workload 1, dual processor. Three pure CPU-bound processes and no I/O at all.

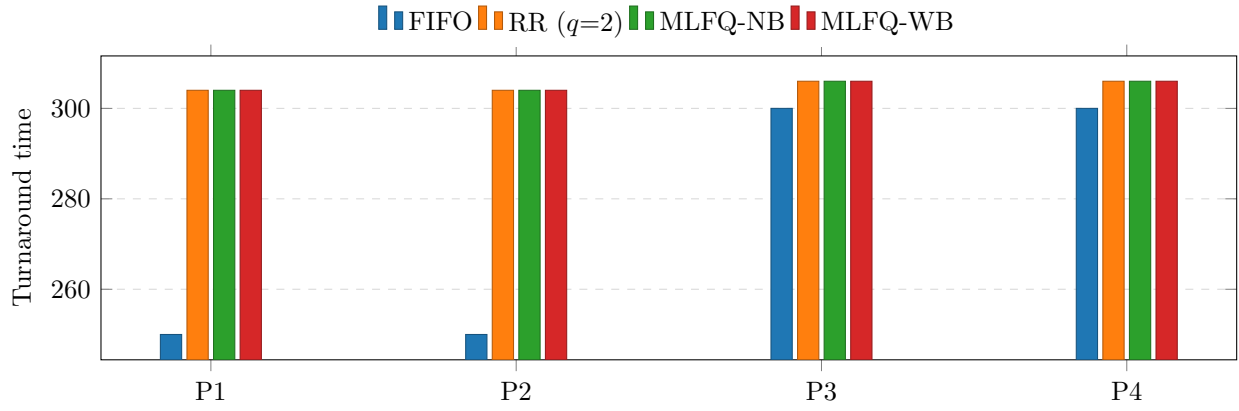


Figure 24: Per-process turnaround time, workload 2, dual processor. Four identical processes with long (50-unit) CPU bursts.

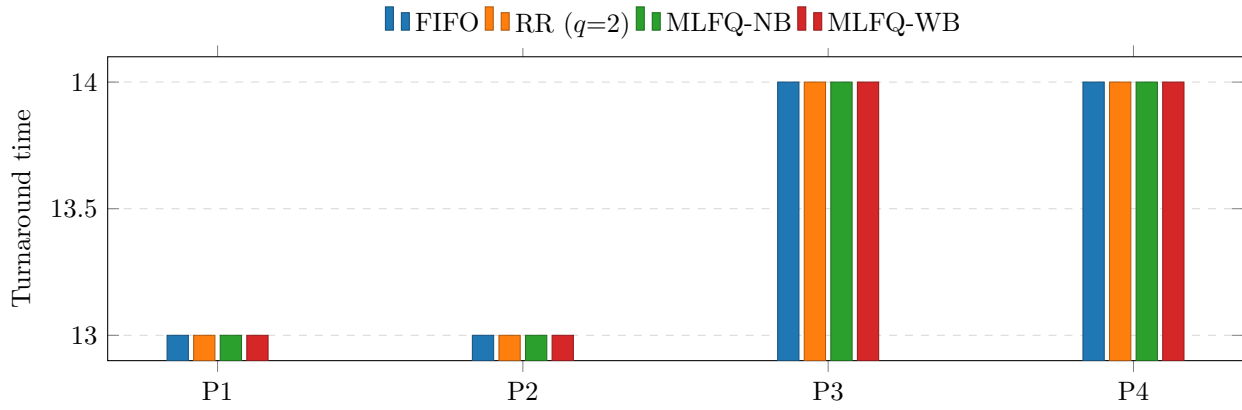


Figure 25: Per-process turnaround time, workload 3, dual processor. Four identical processes whose CPU bursts are all 1 unit long.

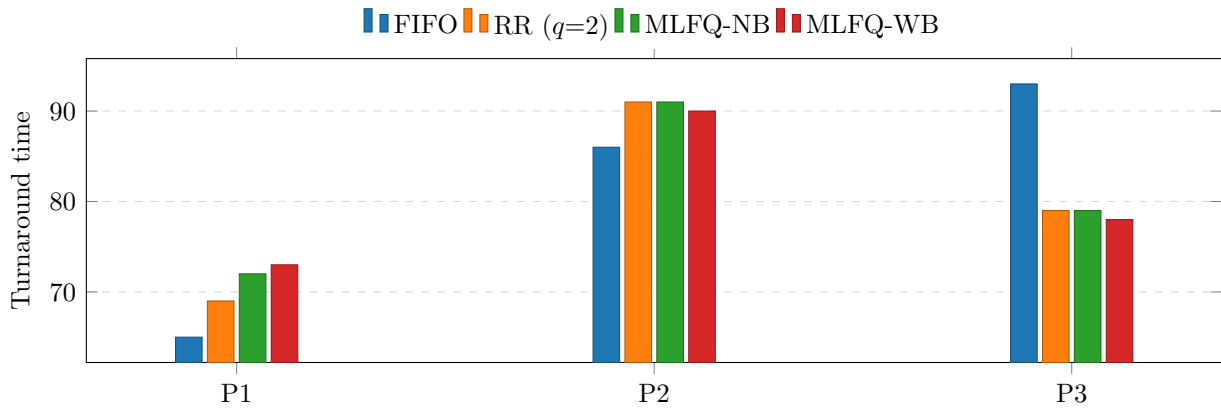


Figure 26: Per-process turnaround time, workload 4, dual processor. P1 long CPU-bound, P2 medium, P3 short interactive.

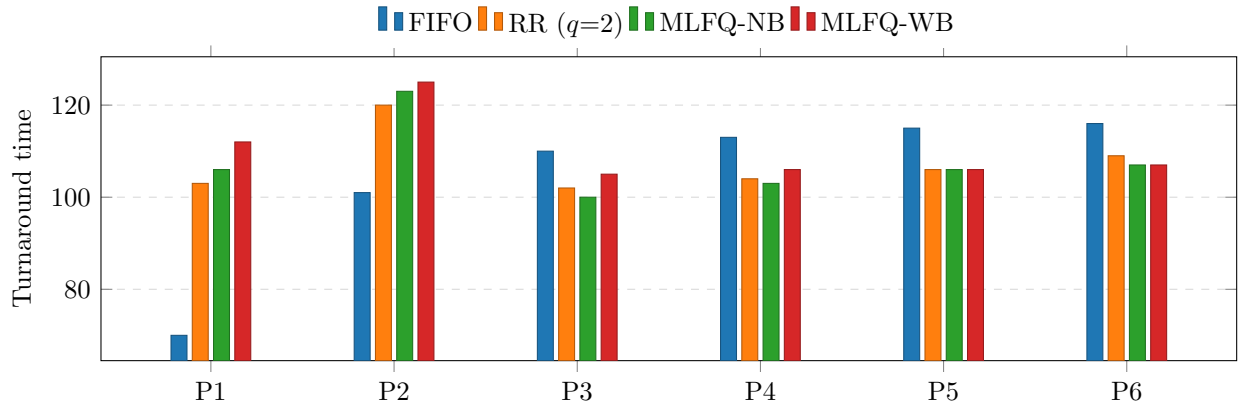


Figure 27: Per-process turnaround time, workload 5, dual processor. P1 long, P2 medium, P3–P6 short interactive, all arriving at $t = 15$.

15.2 Schedule visualisation (Gantt charts)

The charts below show the complete dual-processor schedules. As in Part I each row is a process, but the *colour* of each block now shows which processor ran it: blue for CPU0 and orange for CPU1. A row that changes colour part-way through a burst is a process that migrated between processors. The heading of each chart gives the makespan, the idle time summed over both processors, and the number of migrated bursts.

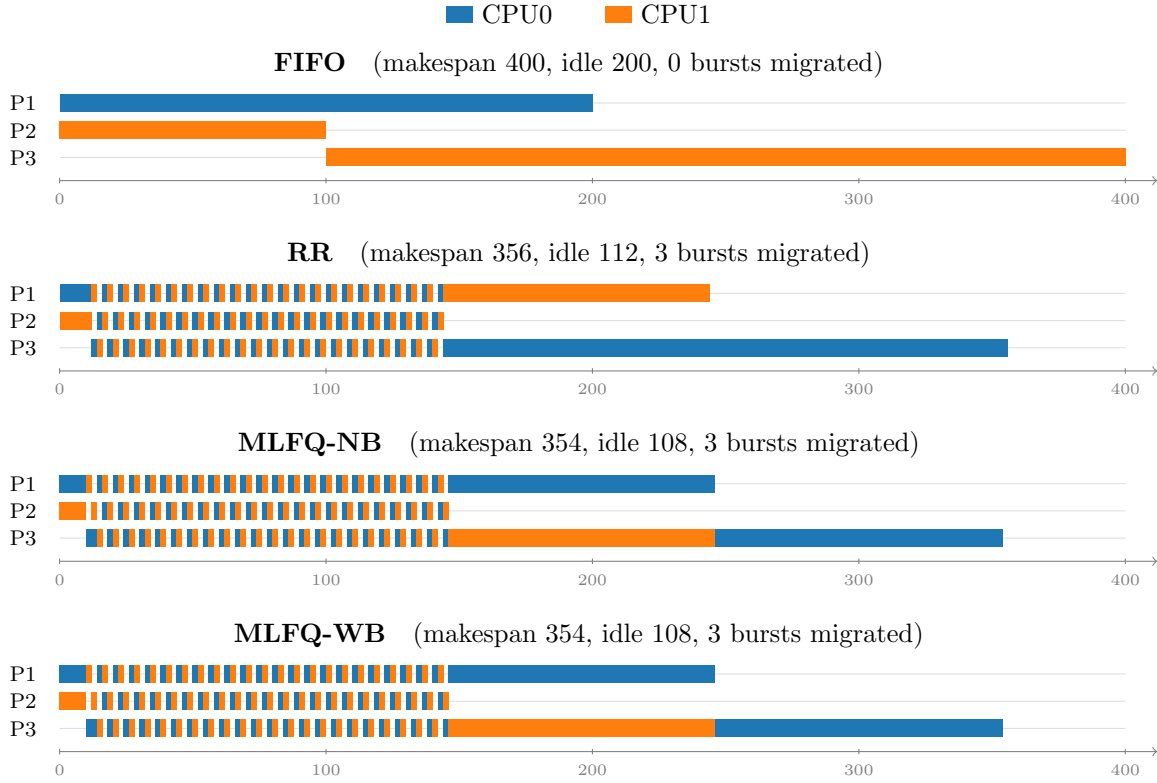


Figure 28: Workload 1, dual processor. Under FIFO, P1 occupies CPU0 for 200 units while P2 and then P3 run on CPU1. Once P1 finishes at 200 only P3 remains, so CPU0 is idle for the last 200 units and the makespan is 400. The preemptive policies keep all three processes alive for longer, so two of them are runnable most of the time; every one of the three bursts migrates between the processors, and the makespan falls to 354–356.

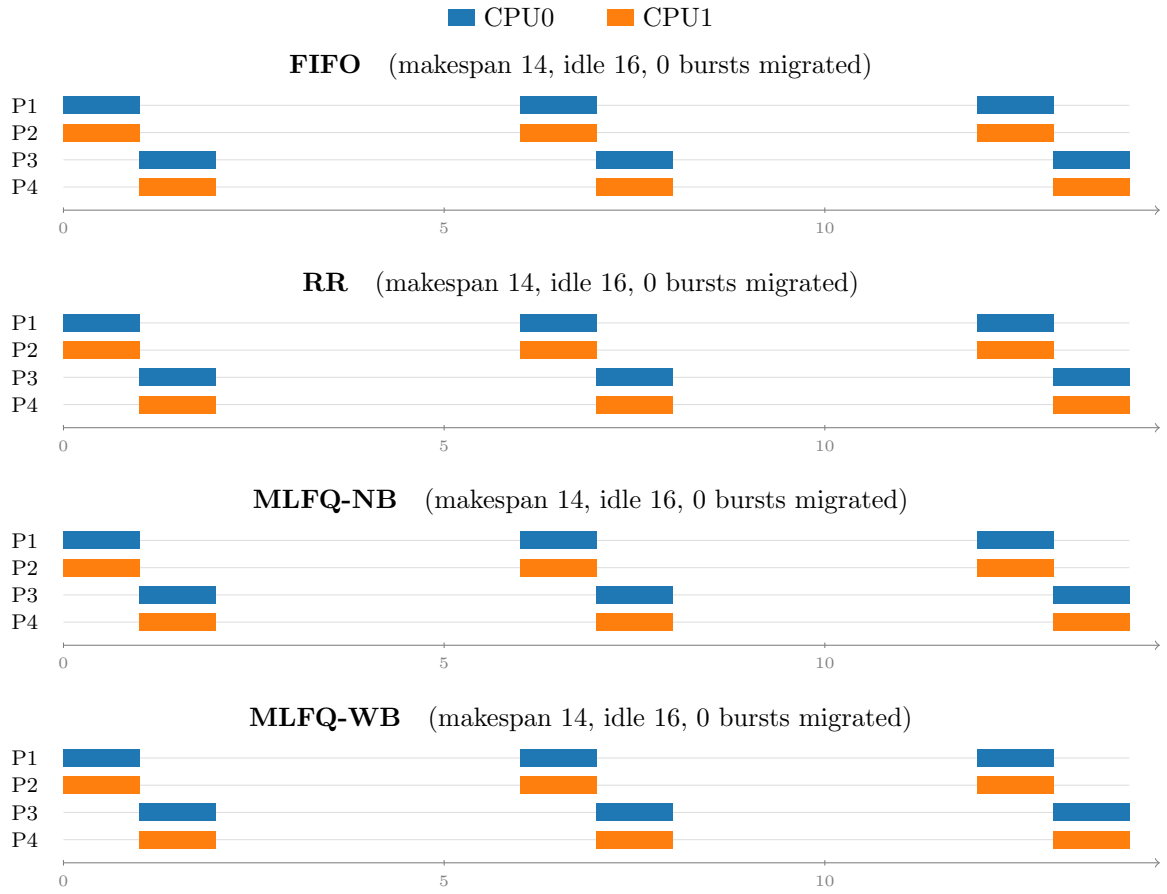


Figure 30: Workload 3, dual processor. All four policies again coincide. The 1-unit bursts are now served two at a time, so the makespan falls to 14, but each processor is busy for only 6 of those 14 units: the workload is limited by I/O, not by processor capacity.

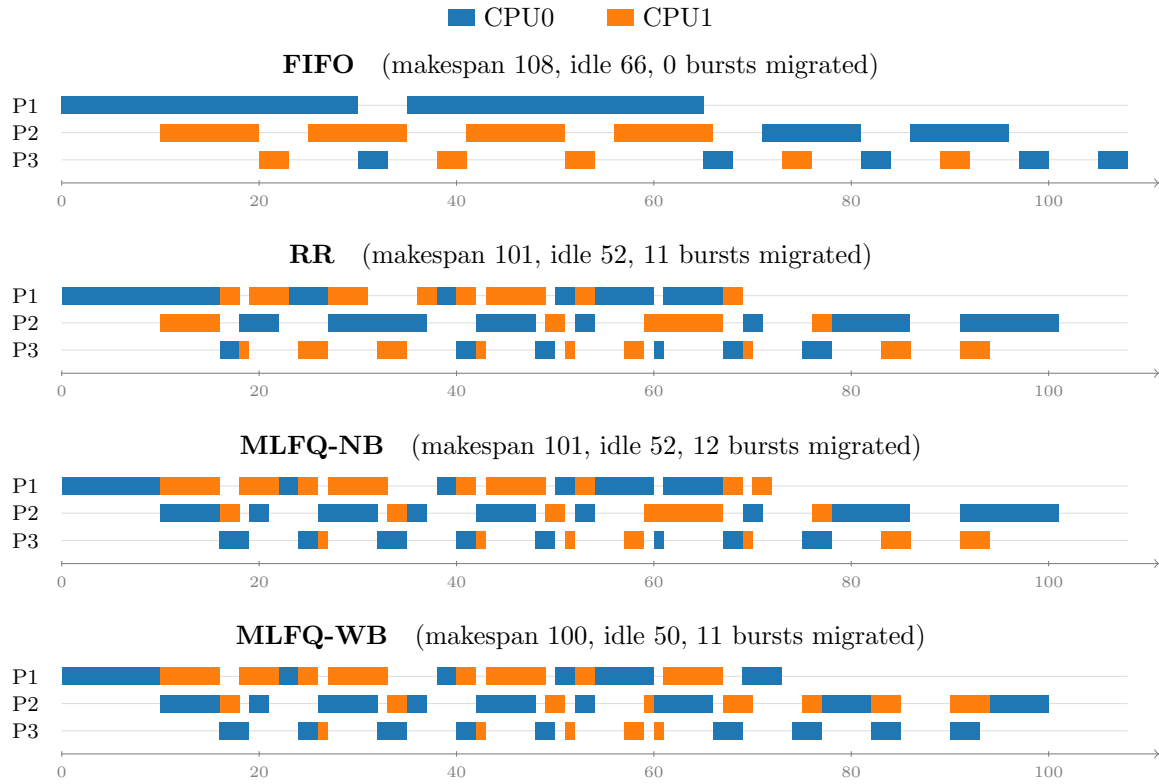


Figure 31: Workload 4, dual processor. Under FIFO the short interactive process P3 no longer has to wait behind the long process P1, because the second processor is often free; P3’s turnaround is 93. Round Robin and the MLFQ variants migrate 11–12 of the 18 CPU bursts between processors and finish at 100–101, against FIFO’s 108.

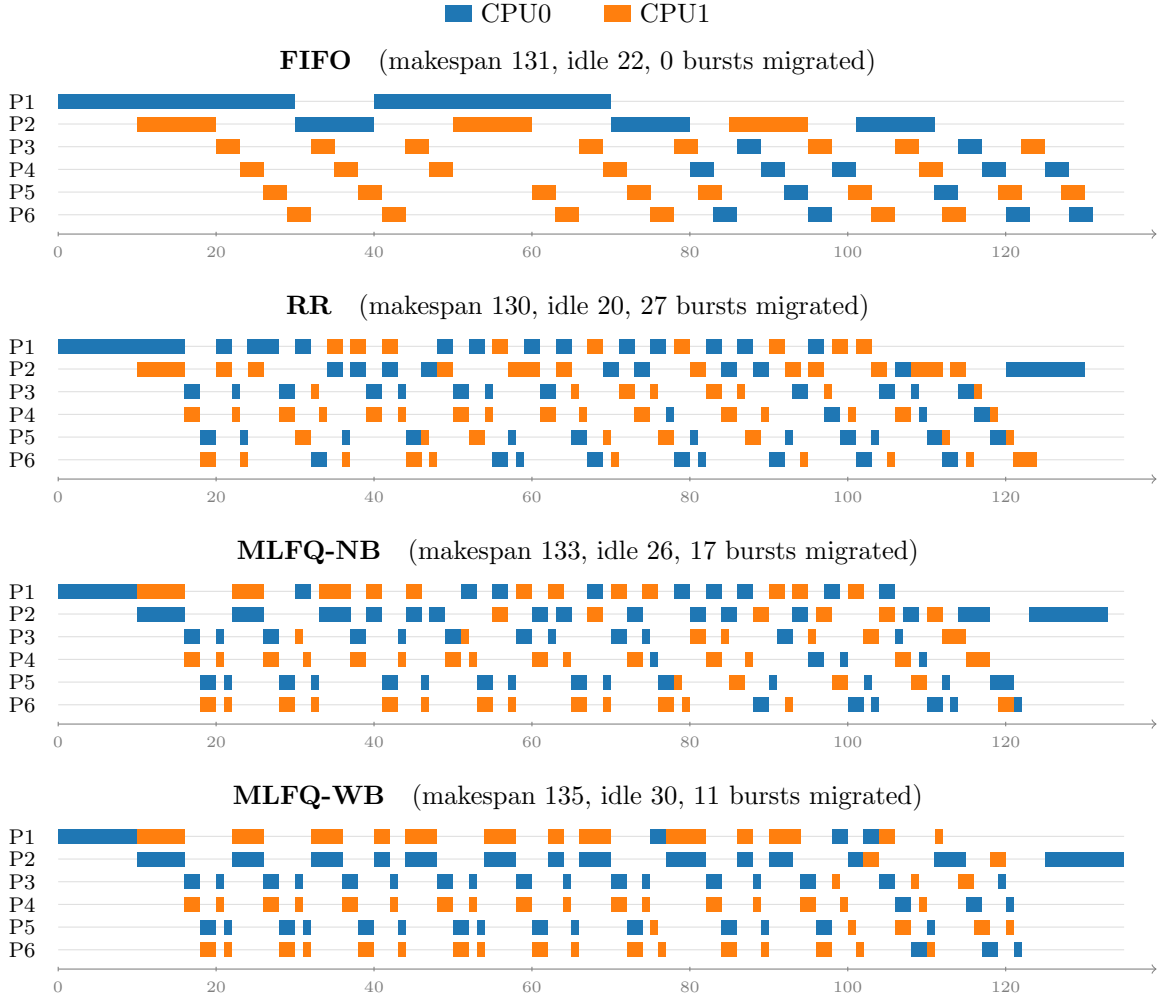


Figure 32: Workload 5, dual processor. FIFO, being non-preemptive, never moves a burst part-way, although 5 of its 6 processes use both processors over their lifetime. Round Robin migrates 27 of the 48 CPU bursts, MLFQ without boost 17 and MLFQ with boost 11.

16 Additional Experiments

16.1 Effect of the Round Robin time quantum

The assignment asks for Round Robin to be studied with different values of the time quantum. The table reports the average turnaround time on the dual processor for $q \in \{1, 2, 4, 8, 16\}$ on all five workloads.

Workload	$q = 1$	$q = 2$	$q = 4$	$q = 8$	$q = 16$
Workload 1	245.00	244.67	244.67	246.00	246.00
Workload 2	307.50	305.00	305.00	305.00	305.00
Workload 3	13.50	13.50	13.50	13.50	13.50
Workload 4	81.67	79.67	80.00	81.00	81.67
Workload 5	107.50	107.33	106.83	105.67	105.33

The best quantum for each workload is shown in bold. Because the absolute turnaround times differ by more than an order of magnitude between workloads, the graph below normalises each workload against its own best quantum, so that all five curves can be compared on a single set of axes.

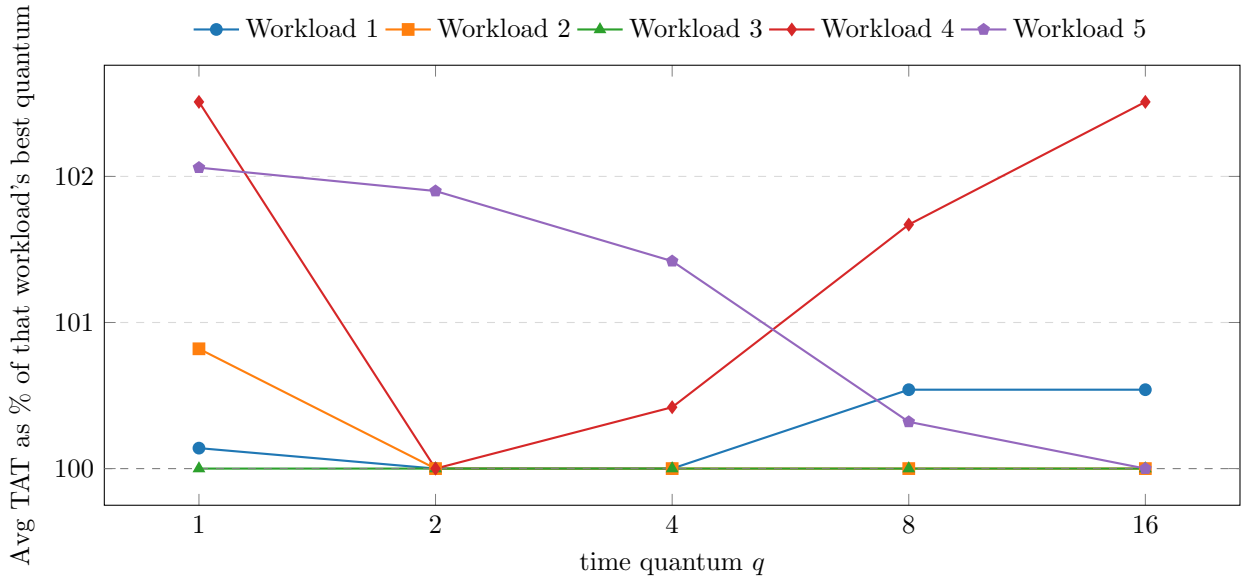


Figure 33: Average turnaround time versus Round Robin time quantum on two processors, all five workloads on one axis, each normalised against its best quantum. Every curve stays within 2.5 % of its best value, far flatter than on one processor (where workload 5 varied by 9.8 %). Workload 5 now improves steadily as the quantum grows, and workload 3 is again perfectly flat.

16.2 Effect of the MLFQ priority boost interval

The assignment asks for a no-boost experiment to be compared against a boost every 20 time units. To place that comparison in context, the boost interval B was swept over $\{0, 5, 10, 20, 40, 80\}$ on the dual processor, where $B = 0$ means boosting is disabled.

Workload	$B = 0$ (off)	$B = 5$	$B = 10$	$B = 20$	$B = 40$	$B = 80$
Workload 1	245.33	245.00	244.67	245.33	245.33	245.33
Workload 2	305.00	294.50	306.50	305.00	305.00	305.00
Workload 3	13.50	13.50	13.50	13.50	13.50	13.50
Workload 4	80.67	80.67	80.33	80.33	80.67	80.67
Workload 5	107.50	107.33	108.17	110.17	112.33	109.17

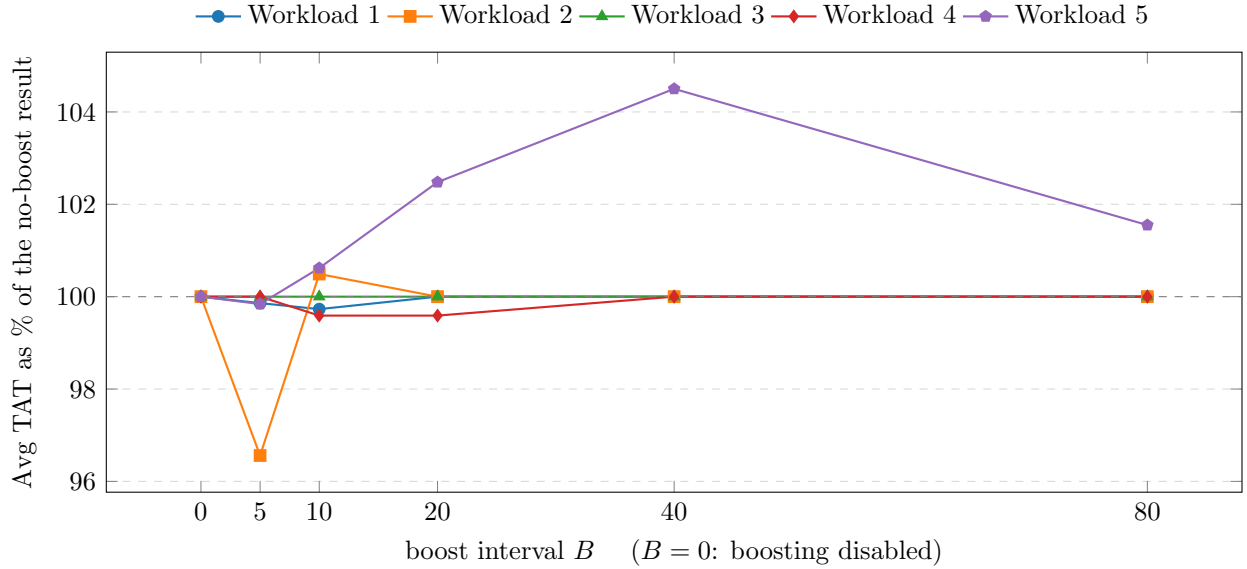


Figure 34: Average turnaround time versus MLFQ priority boost interval on two processors, each workload normalised against its own no-boost result (dashed line). On workload 5 every boost interval of 10 or more makes the average turnaround time *worse* than not boosting at all, the reverse of the single-processor behaviour.

17 Findings and Analysis

17.1 Finding 1: FIFO is no longer best everywhere

On two processors FIFO still produced the lowest average turnaround time on workloads 1, 2 and 5, and tied on workload 3 — but on workload 4 it came *last*:

Workload	FIFO	RR	MLFQ-NB	MLFQ-WB
Workload 1	230.00	244.67	245.33	245.33
Workload 2	275.00	305.00	305.00	305.00
Workload 3	13.50	13.50	13.50	13.50
Workload 4	81.33	79.67	80.67	80.33
Workload 5	104.17	107.33	107.50	110.17

The per-process turnaround times on workload 4 show why:

Workload 4	P1 (long)	P2 (medium)	P3 (short)	Sum
FIFO	65	86	93	244
Round Robin	69	91	79	239

Round Robin makes P1 4 units slower and P2 5 units slower, but makes the interactive P3 **14 units faster**, so its total — and therefore its average — is lower. On one processor the same kind of trade was a bad one for Round Robin (P1 lost 53 units while P3 gained only 37), because every slice given to P3 had to be taken away from P1 or P2. On two processors much of P3’s extra service comes out of processor time that FIFO leaves idle (FIFO idles 66 processor-time units on this workload, Round Robin 52), so the long processes lose far less than P3 gains.

17.2 Finding 2: the convoy effect is much weaker

Maximum TAT	FIFO	RR	MLFQ-NB	MLFQ-WB
Workload 1	390	346	344	344
Workload 2	300	306	306	306
Workload 3	14	14	14	14
Workload 4	93	91	91	90
Workload 5	116	120	123	125

On one processor FIFO’s worst case on workload 4 was far worse than Round Robin’s. On two processors it is only 2 units worse (93 against 91), and the spread between the best- and worst-served process is 28 units under FIFO against 22 under Round Robin. A long burst can still hold one processor, but short processes can now go around it on the other processor, so they are no longer trapped behind it.

On workloads 2 and 5, however, FIFO has the *lower* worst case: interleaving spreads the waiting over every process, which pushes back the process that finishes last.

17.3 Finding 3: where FIFO wastes processor time

Idle time, both CPUs	FIFO	RR	MLFQ-NB	MLFQ-WB
Workload 1	200	112	108	108
Workload 2	0	12	12	12
Workload 3	16	16	16	16
Workload 4	66	52	52	50
Workload 5	22	20	26	30
Utilisation W1	75.0%	84.3%	84.7%	84.7%
Utilisation W2	100.0%	98.0%	98.0%	98.0%
Utilisation W3	42.9%	42.9%	42.9%	42.9%
Utilisation W4	69.4%	74.3%	74.3%	75.0%
Utilisation W5	91.6%	92.3%	90.2%	88.9%

The waste is largest on workload 1, where FIFO leaves 200 processor-time units idle against 108–112 for the preemptive policies. Under FIFO, P1 and P2 finish early, after which P3 is the only process left and can use only one processor. The preemptive policies keep all three processes alive for longer, so two of them are runnable most of the time. On workload 4, FIFO again idles the most (66 units, against 50–52).

Utilisation is lower than on one processor almost everywhere. That is expected: the same amount of work is now spread over twice the processor capacity, and most of these workloads simply cannot keep two processors busy.

The picture is not uniform, however. On workload 2 FIFO is the only policy with *no* idle time at all (0 units, against 12 for the others), and on workload 5 it idles 22 units, between Round Robin’s 20 and MLFQ’s 26–30.

17.4 Finding 4: the priority boost no longer helps

	Workload 4		Workload 5	
	No boost	Boost 20	No boost	Boost 20
Average TAT	80.67	80.33	107.50	110.17
Maximum TAT	91	90	123	125
Minimum TAT	72	73	100	105

On workload 4 the boost now changes the average by only 0.33 units. On workload 5 it makes things clearly *worse* (107.50 → 110.17). Looking at the individual processes on workload 5, the boost makes P1, P2, P3, P4 slower and leaves P5, P6 unchanged; it helps no process at all:

Workload 5	P1	P2	P3	P4	P5	P6
No boost	106	123	100	103	106	107
Boost 20	112	125	105	106	106	107

The boost exists to rescue processes that are starving in the low-priority queues. With two processors, a process sitting in Q_2 still gets a processor whenever no more than one higher-priority process is ready, so there is much less starvation left to fix. The cost of the boost — periodically resetting every process to Q_0 and so discarding what the scheduler has learned — remains, so the net effect turns negative.

17.5 Finding 5: the parameters matter much less, but are still non-monotonic

Across all five workloads, varying the Round Robin quantum from 1 to 16 changes the average turnaround time by at most 2.5% on two processors (on one processor the variation was up to 9.8%). On workload 5 the average now improves steadily as the quantum grows: 107.50, 107.33, 106.83, 105.67, 105.33 for $q = 1, 2, 4, 8, 16$.

The boost interval remains non-monotonic. On workload 2, boosting every 5 units lowers the average from 305.00 to 294.50, but boosting every 10 units raises it to 306.50. As on one processor, these parameters interact with the burst lengths of the specific workload.

17.6 Finding 6: when all four algorithms coincide

Workload 3 again produces identical results for all four policies (average 13.50, maximum 14, minimum 13, makespan 14), for the same reason as in Part I: its 1-unit bursts never reach the quantum. Each processor is busy for only 6 of the 14 time units (42.9% utilisation). This workload is limited by its I/O, so a second processor helps it very little.

17.7 Finding 7: preemption now also causes migration

Bursts migrated	FIFO	RR	MLFQ-NB	MLFQ-WB
Workload 1	0/3	3/3	3/3	3/3
Workload 2	0/12	0/12	0/12	0/12
Workload 3	0/12	0/12	0/12	0/12
Workload 4	0/18	11/18	12/18	11/18
Workload 5	0/48	27/48	17/48	11/48
Dispatches W1	3	136	140	140
Dispatches W2	12	300	300	300
Dispatches W3	12	12	12	12
Dispatches W4	18	40	44	45
Dispatches W5	48	123	120	113

FIFO never migrates a burst, because it never preempts one. The preemptive policies migrate a large fraction of bursts: on workload 5 Round Robin moves 27 of the 48 bursts between processors. No burst migrates on workload 3, because every burst lasts a single unit, or on workload 2, where the processes always re-enter the shared queue in the same order and so each one is always dispatched to the same processor. The simulator treats migration as free, but on real hardware a migrated process loses its warm cache, which is why real operating systems try to keep a process on the same processor.

17.8 Finding 8: the two processors are not equally loaded

CPU0 / CPU1 busy time	FIFO	RR	MLFQ-NB	MLFQ-WB
Workload 1	200 / 400	356 / 244	354 / 246	354 / 246
Workload 2	300 / 300	300 / 300	300 / 300	300 / 300
Workload 3	6 / 6	6 / 6	6 / 6	6 / 6
Workload 4	95 / 55	90 / 60	90 / 60	91 / 59
Workload 5	129 / 111	130 / 110	132 / 108	131 / 109

Because CPU0 always dispatches first, it receives the work whenever only one process is ready, and on workload 4 it is busy 95 units against CPU1's 55 under FIFO. The imbalance can also go the other way: on workload 1 under FIFO the 300-unit process P3 happens to land on CPU1, which ends up busy for 400 units against CPU0's 200. Workloads 2 and 3 are perfectly balanced under every policy. With a shared queue this imbalance does not cost any turnaround time — neither processor is ever idle while work is waiting — but with per-processor queues it would, and a real scheduler would need a load-balancing policy.

18 Summary of Part II

Policy	On two processors	Change from one processor
FIFO	Lowest average turnaround on W1, W2 and W5; never migrates; fewest dispatches	Loses the average-turnaround lead on W4; still idles the processors most on W1 and W4; the convoy effect is much weaker
Round Robin	Lowest average turnaround on W4; lower worst case than FIFO on W1 and W4	Becomes the best policy on W4; less sensitive to the quantum on most workloads; migrates many bursts
MLFQ (no boost)	Close to Round Robin everywhere; better than the boosted variant on W5	Differences from Round Robin shrink to a few time units
MLFQ (boost 20)	Worst average turnaround on W5	The boost stops paying off: negligible on W4 and harmful on W5

Part III

Single versus Dual Processor

19 Head-to-Head Results

This part places the results of Part I and Part II side by side. For every workload and algorithm, the “1” columns are from the single-processor simulator and the “2” columns from the dual-processor simulator. Δ is the reduction obtained by adding the second processor, as a percentage of the single-processor value; a positive Δ is an improvement.

19.1 Turnaround time

W	Algorithm	Average TAT			Maximum TAT			Minimum TAT	
		1	2	Δ	1	2	Δ	1	2
W1	FIFO	363.33	230.00	36.7%	590	390	33.9%	200	100
	RR	458.67	244.67	46.7%	590	346	41.4%	294	144
	MLFQ-NB	461.33	245.33	46.8%	590	344	41.7%	298	146
	MLFQ-WB	461.33	245.33	46.8%	590	344	41.7%	298	146
W2	FIFO	525.00	275.00	47.6%	600	300	50.0%	450	250
	RR	595.00	305.00	48.7%	600	306	49.0%	588	304
	MLFQ-NB	595.00	305.00	48.7%	600	306	49.0%	588	304
	MLFQ-WB	597.00	305.00	48.9%	600	306	49.0%	594	304
W3	FIFO	14.50	13.50	6.9%	16	14	12.5%	13	13
	RR	14.50	13.50	6.9%	16	14	12.5%	13	13
	MLFQ-NB	14.50	13.50	6.9%	16	14	12.5%	13	13
	MLFQ-WB	14.50	13.50	6.9%	16	14	12.5%	13	13
W4	FIFO	123.00	81.33	33.9%	163	93	42.9%	73	65
	RR	131.33	79.67	39.3%	142	91	35.9%	126	69
	MLFQ-NB	135.33	80.67	40.4%	144	91	36.8%	127	72
	MLFQ-WB	131.67	80.33	39.0%	147	90	38.8%	112	73
W5	FIFO	189.00	104.17	44.9%	225	116	48.4%	82	70
	RR	215.67	107.33	50.2%	234	120	48.7%	206	102
	MLFQ-NB	212.83	107.50	49.5%	233	123	47.2%	200	100
	MLFQ-WB	206.17	110.17	46.6%	233	125	46.4%	186	105

19.2 Makespan, utilisation and overhead

W	Algorithm	Makespan			Utilisation (%)		Idle time		Dispatches	
		1	2	Speedup	1	2	1	2	1	2
W1	FIFO	600	400	1.50×	100.0	75.0	0	200	3	3
	RR	600	356	1.69×	100.0	84.3	0	112	247	136
	MLFQ-NB	600	354	1.69×	100.0	84.7	0	108	248	140
	MLFQ-WB	600	354	1.69×	100.0	84.7	0	108	248	140
W2	FIFO	600	300	2.00×	100.0	100.0	0	0	12	12
	RR	600	306	1.96×	100.0	98.0	0	12	300	300
	MLFQ-NB	600	306	1.96×	100.0	98.0	0	12	300	300
	MLFQ-WB	600	306	1.96×	100.0	98.0	0	12	300	300
W3	FIFO	16	14	1.14×	75.0	42.9	4	16	12	12
	RR	16	14	1.14×	75.0	42.9	4	16	12	12
	MLFQ-NB	16	14	1.14×	75.0	42.9	4	16	12	12
	MLFQ-WB	16	14	1.14×	75.0	42.9	4	16	12	12
W4	FIFO	178	108	1.65×	84.3	69.4	28	66	18	18
	RR	152	101	1.50×	98.7	74.3	2	52	68	40
	MLFQ-NB	154	101	1.52×	97.4	74.3	4	52	67	44
	MLFQ-WB	157	100	1.57×	95.5	75.0	7	50	62	45
W5	FIFO	240	131	1.83×	100.0	91.6	0	22	48	48
	RR	244	130	1.88×	98.4	92.3	4	20	131	123
	MLFQ-NB	243	133	1.83×	98.8	90.2	3	26	131	120
	MLFQ-WB	243	135	1.80×	98.8	88.9	3	30	131	113

Idle time on two processors is summed over both processors.

19.3 Ranking of the algorithms

Algorithms ordered from lowest (best) to highest average turnaround time; “=” marks a tie.

Workload	One processor	Two processors
W1	FIFO < RR < MLFQ-NB = MLFQ-WB	FIFO < RR < MLFQ-NB = MLFQ-WB
W2	FIFO < RR = MLFQ-NB < MLFQ-WB	FIFO < RR = MLFQ-NB = MLFQ-WB
W3	FIFO = RR = MLFQ-NB = MLFQ-WB	FIFO = RR = MLFQ-NB = MLFQ-WB
W4	FIFO < RR < MLFQ-WB < MLFQ-NB	RR < MLFQ-WB < MLFQ-NB < FIFO
W5	FIFO < MLFQ-WB < MLFQ-NB < RR	FIFO < RR < MLFQ-NB < MLFQ-WB

19.4 How much the choice of algorithm matters

The *spread* is the gap between the best and worst algorithm’s average turnaround time, as a percentage of the best. The *quantum sensitivity* is the gap between the best and worst Round Robin quantum in the sweep, as a percentage of the best. The *boost effect* is the change in average turnaround time when MLFQ’s boost is turned on (negative means the boost helped).

Workload	Spread		Quantum sensitivity		Boost effect	
	1 CPU	2 CPUs	1 CPU	2 CPUs	1 CPU	2 CPUs
W1	27.0%	6.7%	1.2%	0.5%	+0.00%	+0.00%
W2	13.7%	10.9%	0.6%	0.8%	+0.34%	+0.00%
W3	0.0%	0.0%	0.0%	0.0%	+0.00%	+0.00%
W4	10.0%	2.1%	2.6%	2.5%	-2.71%	-0.41%
W5	14.1%	5.8%	9.8%	2.1%	-3.13%	+2.48%

20 Comparative Graphs

Every chart in this section shows all five workloads on a single axis. Where the absolute values of the workloads differ too much to share an axis, the chart shows a relative quantity instead: a reduction, a speedup or a percentage-point change.

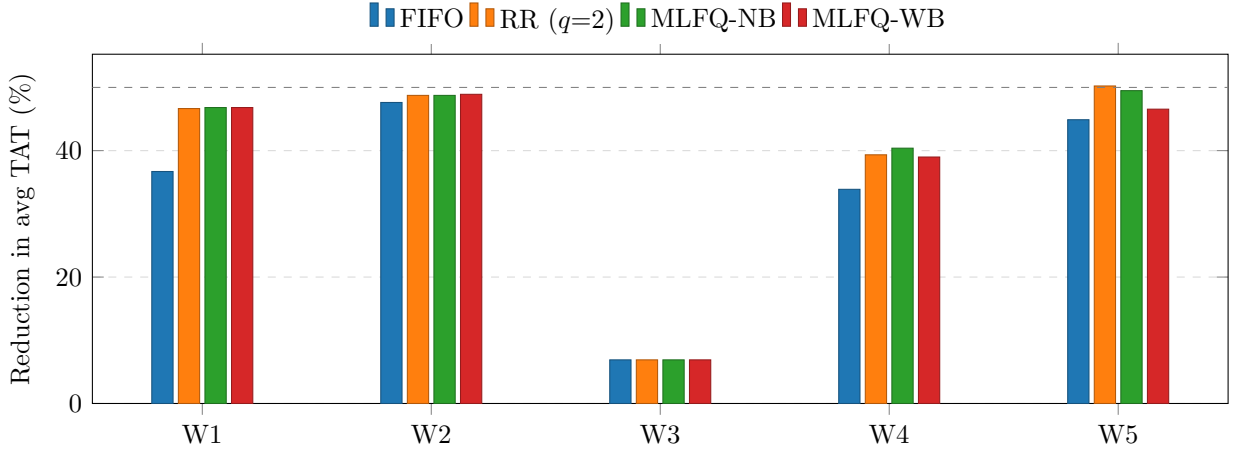


Figure 35: Reduction in average turnaround time obtained by adding a second processor. The dashed line at 50 % would correspond to halving the turnaround time. Every policy improves on every workload, and on workloads 1, 2, 4 and 5 the preemptive policies improve by more than FIFO.

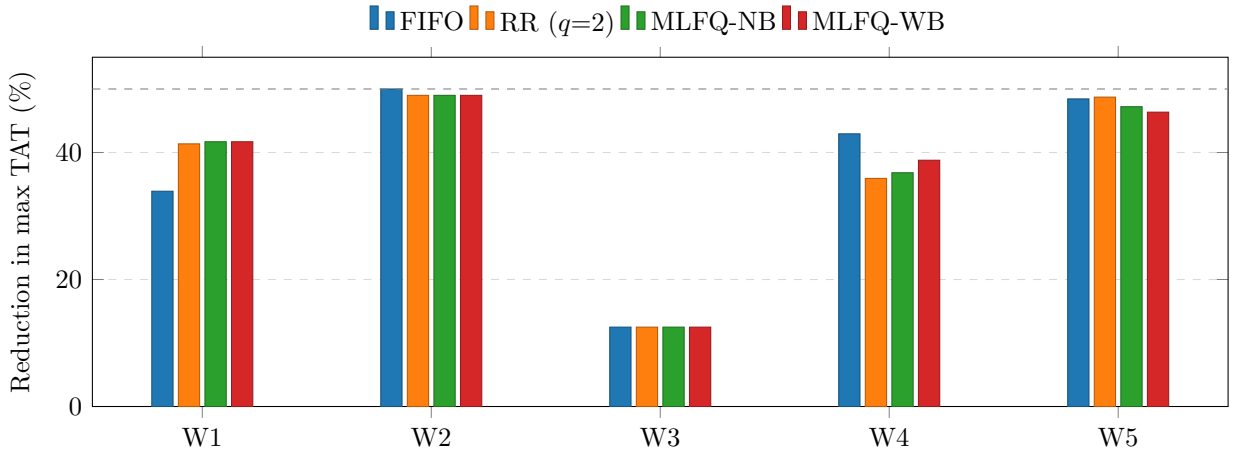


Figure 36: Reduction in maximum (worst-case) turnaround time obtained by adding a second processor.

20.1 Per-process improvement

The next five charts show, for each process, how much its turnaround time was reduced by the second processor.

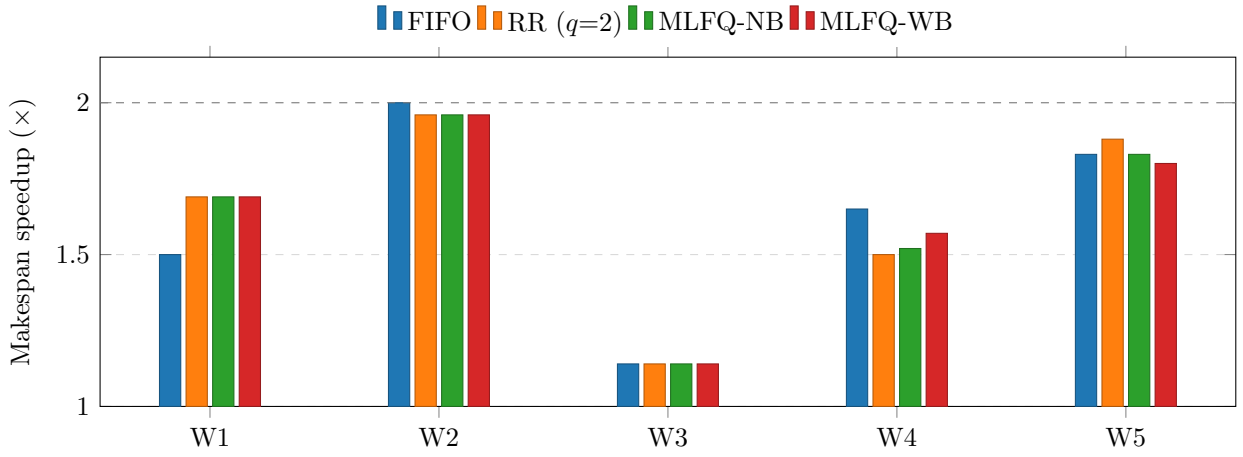


Figure 37: Makespan speedup, single-processor makespan divided by dual-processor makespan. The dashed line at $2\times$ is the theoretical maximum for two processors; only FIFO on workload 2 reaches it.

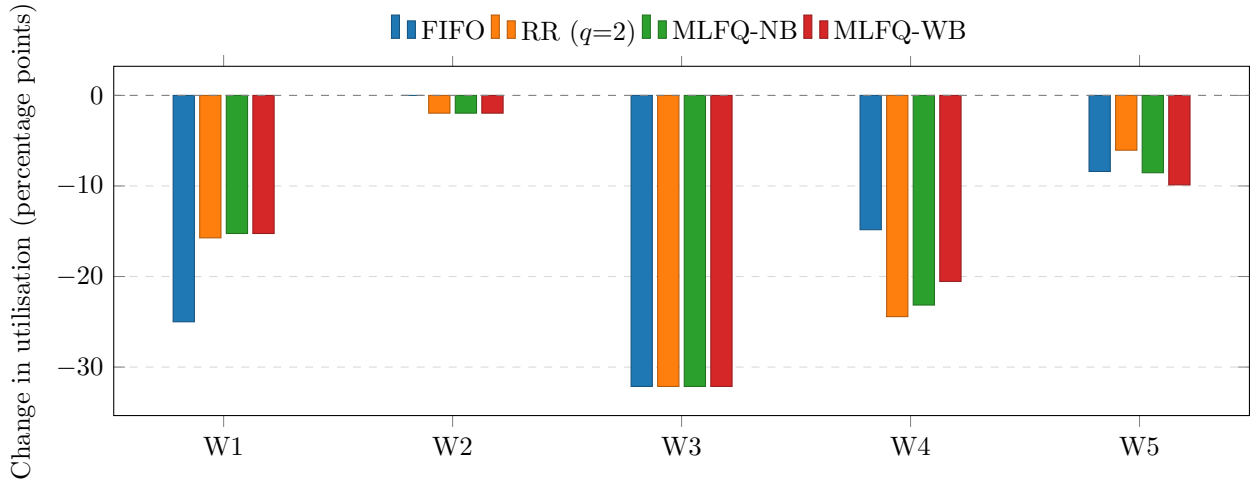


Figure 38: Change in CPU utilisation when the second processor is added (dual minus single, in percentage points). Utilisation falls almost everywhere, because the same work is spread over twice the processor capacity.

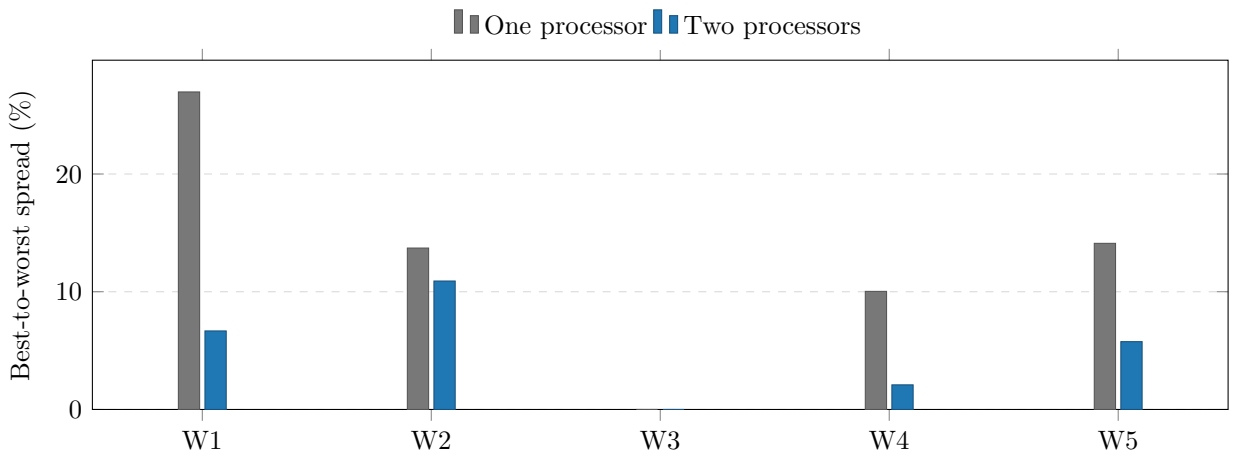


Figure 39: Spread between the best and worst algorithm's average turnaround time, as a percentage of the best, on one and two processors. The choice of algorithm matters much less with two processors.

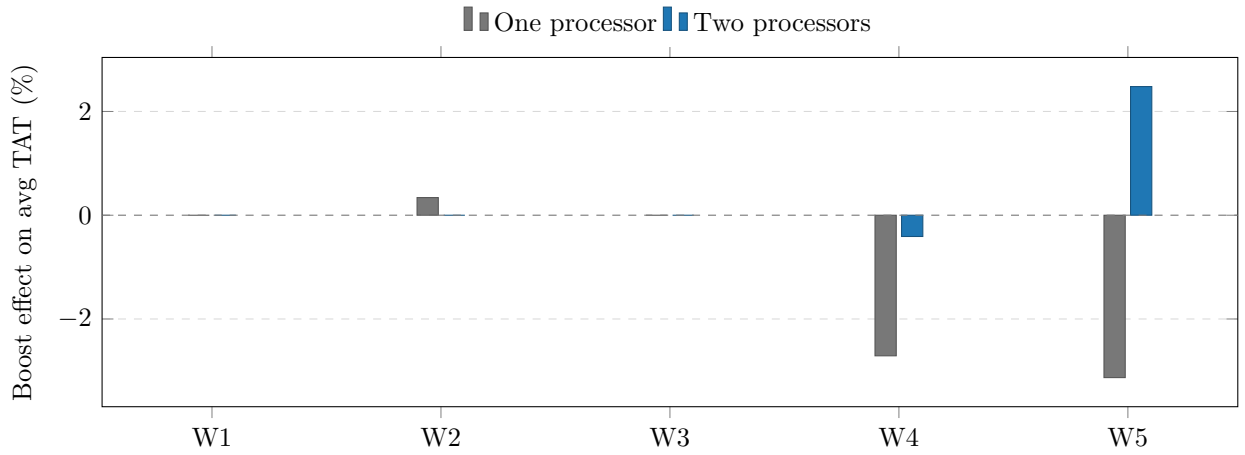


Figure 40: Effect of turning MLFQ’s priority boost on, as the percentage change in average turnaround time (negative = the boost helped). On one processor the boost helps workloads 4 and 5; on two processors it barely changes workload 4 and hurts workload 5.

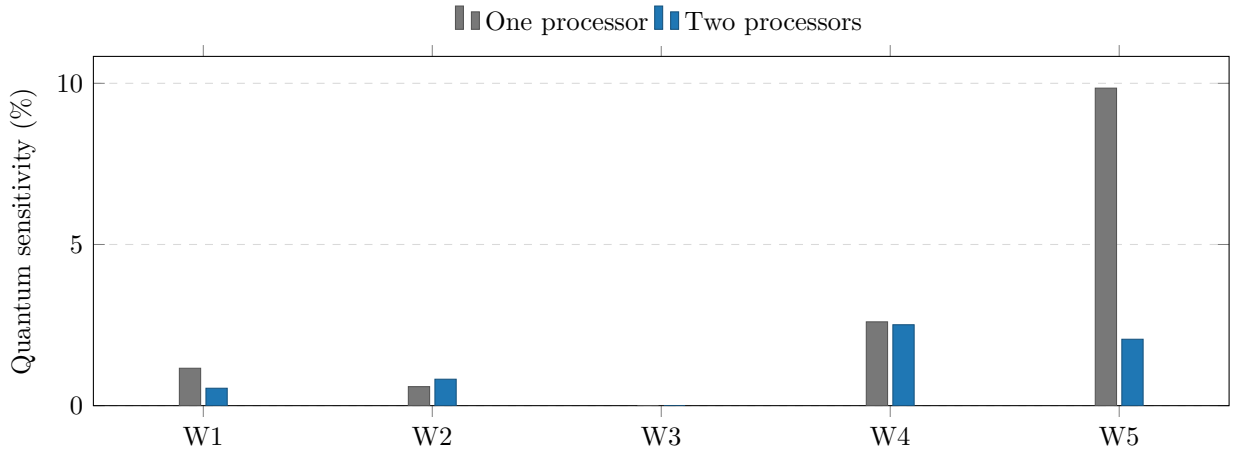


Figure 41: Sensitivity of Round Robin to its time quantum: the gap between the best and worst quantum in the sweep $q \in \{1, 2, 4, 8, 16\}$, as a percentage of the best.

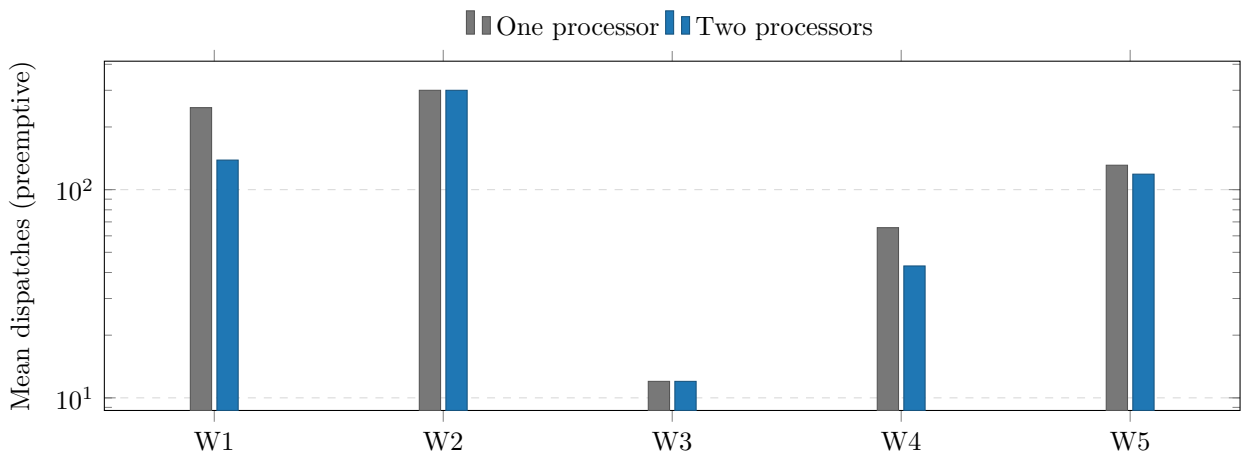


Figure 42: Mean number of CPU dispatches of the three preemptive policies, on one and two processors (logarithmic scale).

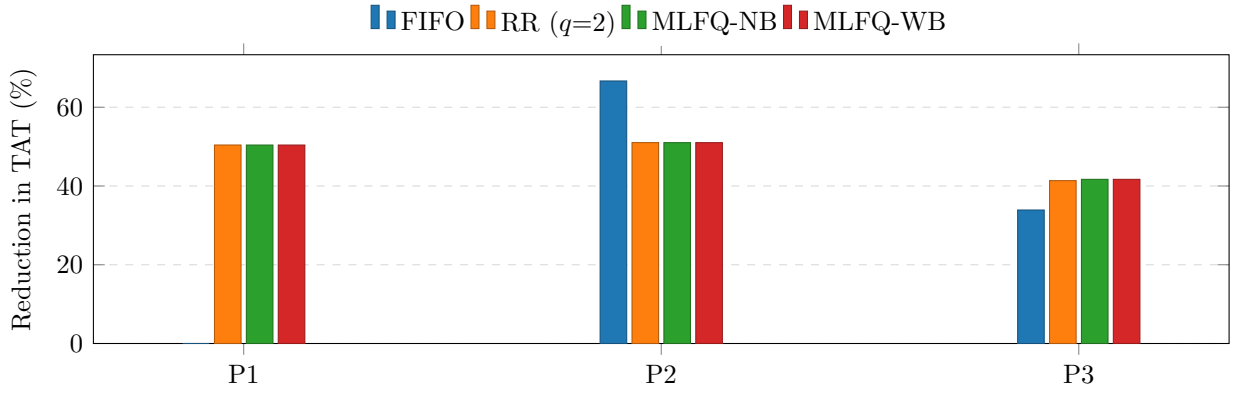


Figure 43: Reduction in per-process turnaround time from one to two processors, workload 1.

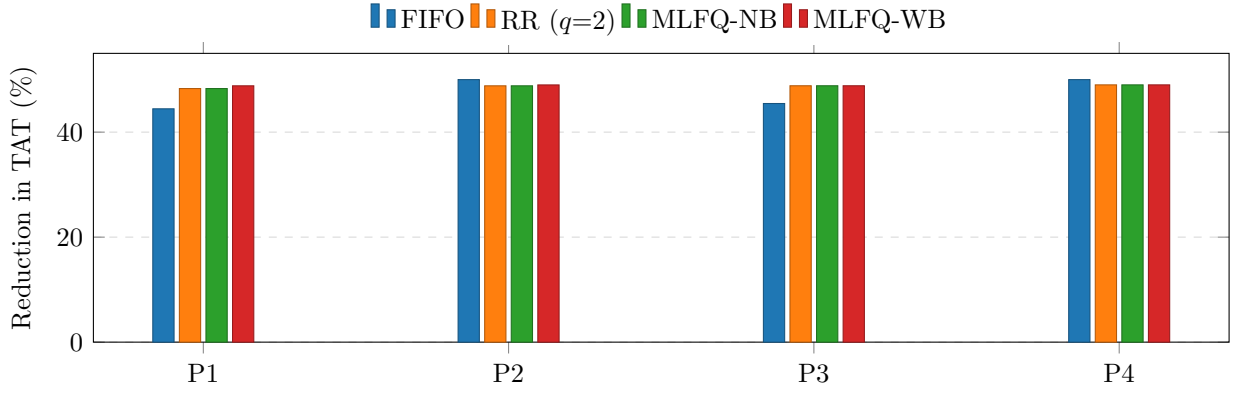


Figure 44: Reduction in per-process turnaround time from one to two processors, workload 2.

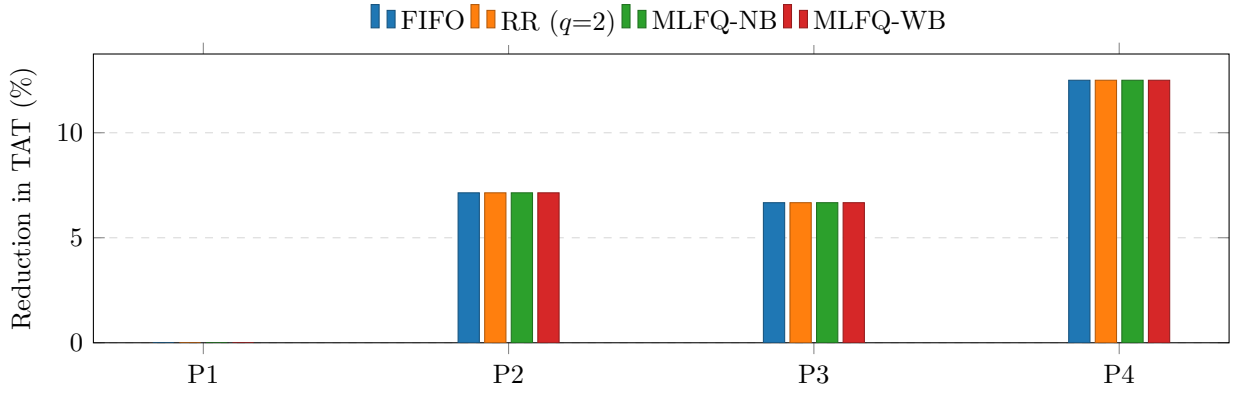


Figure 45: Reduction in per-process turnaround time from one to two processors, workload 3.

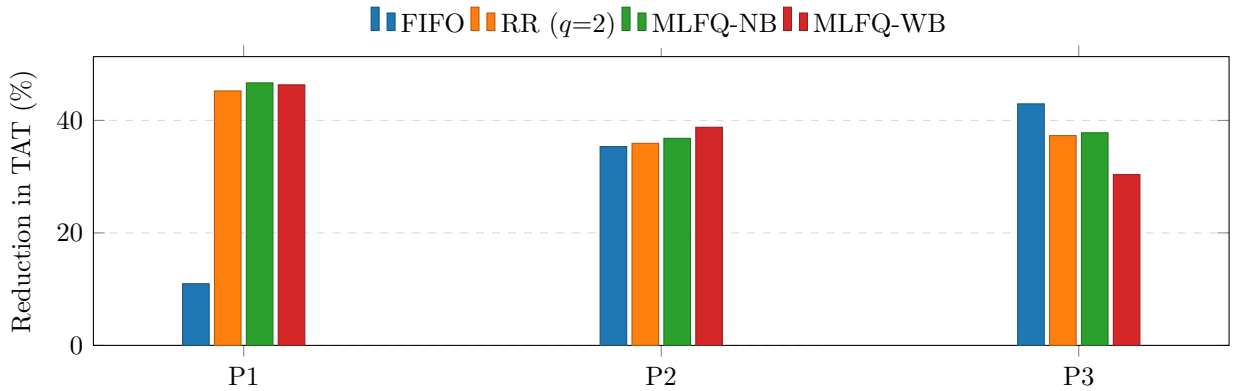


Figure 46: Reduction in per-process turnaround time from one to two processors, workload 4.

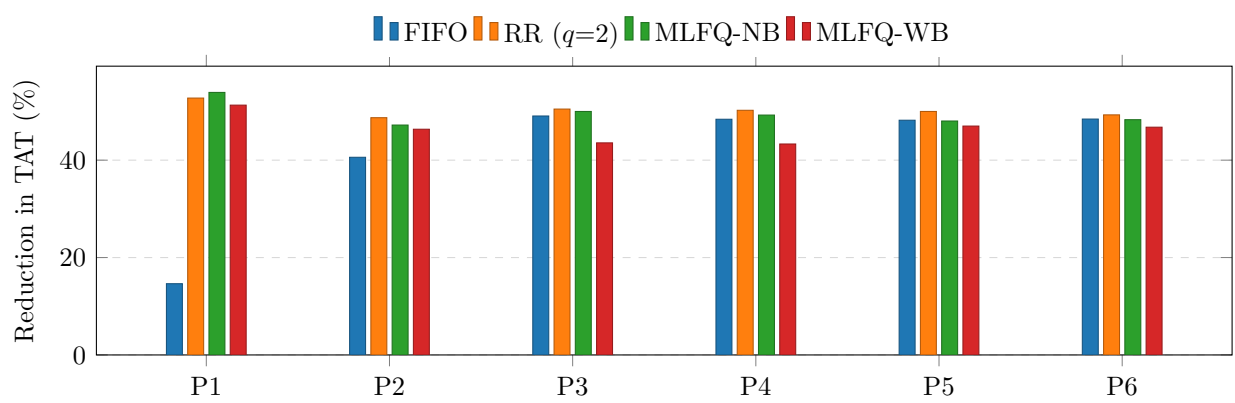


Figure 47: Reduction in per-process turnaround time from one to two processors, workload 5.

20.2 The same workload on one and two processors

Figure 48 shows workload 4 under FIFO and Round Robin on one processor and on two processors, on a common time axis. Colour shows the processor that ran each block.

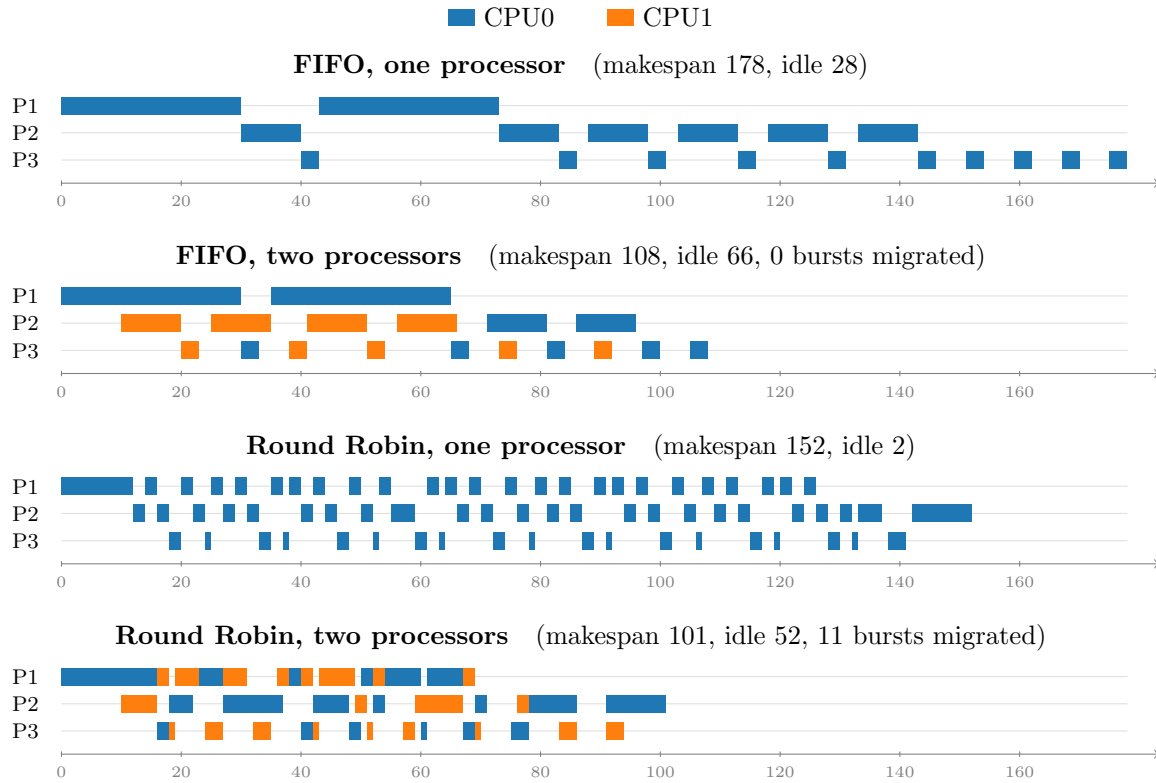


Figure 48: Workload 4 on one and two processors. Under FIFO the short process P3 finishes at 178 on one processor but at 108 on two, because it no longer queues behind the long process P1. Round Robin's makespan falls from 152 to 101.

21 Findings of the Comparison

21.1 A second processor helps every policy on every workload

Average turnaround time fell in all 20 algorithm/workload combinations. The reduction ranges from 6.9% (FIFO on workload 3) to 50.2% (RR on workload 5). The gains are largest on workloads 2 and 5, where at least two processes want the processor most of the time, and smallest on workload 3, whose processes spend most of their time blocked on I/O — which no extra processor can speed up.

21.2 The speedup is almost always below 2

Two processors can at most halve the makespan. That happened only once, for FIFO on workload 2, where four identical CPU-bound processes pair up perfectly on the two processors. Everywhere else the speedup was lower, for exactly the reasons given in Section 3.5:

- **Workload 1** (speedup 1.50–1.69 $\times$): there are only three processes, and once two have finished the last one can use only one processor. Under FIFO the second processor is idle for the final 200 time units.
- **Workload 3** (speedup 1.14 $\times$): the processes are I/O-bound, and each processor is idle for most of the run.
- **Workloads 4 and 5** (speedup 1.50–1.88 $\times$): a mixture of both — few processes, and long I/O waits.

21.3 The preemptive policies gain more than FIFO

On every workload except workload 3 (where all four policies are identical), each preemptive policy’s average turnaround time fell by more than FIFO’s:

Reduction in avg TAT	FIFO	RR	MLFQ-NB	MLFQ-WB
Workload 1	36.7%	46.7%	46.8%	46.8%
Workload 2	47.6%	48.7%	48.7%	48.9%
Workload 3	6.9%	6.9%	6.9%	6.9%
Workload 4	33.9%	39.3%	40.4%	39.0%
Workload 5	44.9%	50.2%	49.5%	46.6%

This is the key interaction between the policy and the number of processors. The main weakness of the preemptive policies on one processor was that interleaving delays every process: each process has to share one processor with all the others. With two processors each processor is shared by fewer processes, so that penalty shrinks. FIFO had no such penalty to begin with, so it has less to gain.

21.4 The ranking changes and the gaps narrow

On workload 4 FIFO moves from first place to last: on one processor the order is FIFO < RR < MLFQ-WB < MLFQ-NB, while on two it is RR < MLFQ-WB < MLFQ-NB < FIFO. On workload 5 the boosted MLFQ moves from second place to last. The spread between the best and worst policy narrowed on every workload that has one: from 27.0% to 6.7% on workload 1, from 13.7% to 10.9% on workload 2, from 10.0% to 2.1% on workload 4 and from 14.1% to 5.8% on workload 5. With more processing capacity, the order in which processes are served matters less, so the choice of scheduling policy matters less.

21.5 Utilisation falls: diminishing returns

Utilisation fell in 19 of the 20 combinations; the only exception is FIFO on workload 2, which kept both processors busy all the time. The largest drop was on workload 3, from 75.0 % to 42.9 %. The second processor adds capacity, but these workloads cannot supply enough runnable work to use it fully. This is the diminishing return of adding processors: each extra processor helps less than the last, because an ever larger share of it stands idle.

21.6 The convoy effect largely disappears

On one processor FIFO's worst case on workload 4 was 21 units worse than Round Robin's (163 against 142). On two processors the gap is 2 units (93 against 91). The interactive process P3's turnaround under FIFO fell from 163 to 93 — a 43 % improvement, the largest of any process on that workload — because it no longer has to wait for the long process P1 to release the only processor.

21.7 The priority boost stops paying off

On one processor the boost improved MLFQ's average turnaround time by 2.71 % on workload 4 and 3.13 % on workload 5. On two processors it improved workload 4 by only 0.41 % and made workload 5 2.48 % *worse*. The boost is a remedy for starvation in the low-priority queues; with a second processor there is much less starvation to remedy, while the boost's cost — discarding the priority information MLFQ has learned — is unchanged.

21.8 Parameter tuning matters less

Round Robin became less sensitive to its time quantum on workloads 1, 4 and 5. The change is largest on workload 5, where the gap between the best and worst quantum fell from 9.8 % to 2.1 %. Workload 2 is a small exception, rising from 0.6 % to 0.8 %, and workload 3 is insensitive on both.

21.9 Fewer dispatches, but a new overhead

FIFO's dispatch count is unchanged on every workload. The preemptive policies need fewer dispatches on workloads 1, 4 and 5 — Round Robin on workload 1 falls from 247 to 136 — because with fewer competitors per processor, a preempted process is more often immediately re-dispatched on the same processor, which extends its segment instead of starting a new one. In exchange, the second processor introduces migration: the preemptive policies moved up to 27 bursts between processors on workload 5, a cost that does not exist on a single processor.

22 Overall Summary and Conclusion

Policy	One processor	Two processors	Effect of the second processor
FIFO	Best average TAT on every workload (tied on W3); highest worst-case turnaround and lowest utilisation on W4	Best average TAT on W1, W2, W5; worst on W4	Smallest gains (6.9–47.6 %); only policy to reach 2× speedup
Round Robin	Fairest on W4 and W5; best utilisation on W4; worse average than FIFO	Best average TAT on W4	Gains of 6.9–50.2 %, including the largest of all (W5); less sensitive to q on most workloads
MLFQ (no boost)	Learns process behaviour; average close to Round Robin	Very close to Round Robin	Gains of 6.9–49.5 %
MLFQ (boost 20)	Boost improves average on W4 and W5	Worst average TAT on W5	The boost stops helping

Conclusion

The experiments lead to three main conclusions.

No policy is best on every metric. On a single processor FIFO had the lowest average turnaround time on every workload (tied on workload 3), yet on workload 4 it was the worst policy for worst-case turnaround, fairness and CPU utilisation. The preemptive policies reversed that trade-off, and MLFQ approached Round Robin’s fairness while automatically identifying interactive processes, with the boost removing its starvation weakness.

A second processor helps everyone, but never doubles performance for free. Average turnaround time improved by 6.9–50.2 % and the makespan by 1.14–2.00×. The speedup reached 2× only in the ideal case of identical CPU-bound processes; with few processes or significant I/O, much of the second processor stood idle.

The second processor changes which policy is preferable. It weakens the very weaknesses that distinguished the policies on one processor: FIFO’s convoy effect largely disappears, the preemptive policies’ interleaving penalty shrinks, the boost — a remedy for starvation in MLFQ’s lower queues — no longer pays off, and Round Robin becomes much less sensitive to its quantum on most workloads. The policies therefore end up much closer together, and on workload 4 the winner changes from FIFO to Round Robin. A scheduling policy tuned for a single processor cannot be assumed to remain the best choice when the number of processors changes.